CS336 Assignment 1 (basics): Building a Transformer LM

CS336 作业 1（基础）：构建 Transformer 语言模型

Version 26.0.3

版本 26.0.3

CS336 Staff

CS336 教学团队

Spring 2026

2026 年春季

---

1 Assignment Overview

作业概览

In this assignment, you will build all the components needed to train a standard Transformer language model (LM) from scratch and train some models.

在本作业中，你将从头构建训练标准 Transformer 语言模型（LM）所需的所有组件，并训练一些模型。

What you will implement

你将实现的内容

1. Byte-pair encoding (BPE) tokenizer (Section 2)

   字节对编码（BPE）分词器（第 2 节）

2. Transformer language model (LM) (Section 3)

   Transformer 语言模型（LM）（第 3 节）

3. The cross-entropy loss function and the AdamW optimizer (Section 4)

   交叉熵损失函数和 AdamW 优化器（第 4 节）

4. The training loop, with support for serializing and loading model and optimizer state (Section 5)

   训练循环，支持序列化和加载模型及优化器状态（第 5 节）

What you will run

你将运行的内容

1. BPE tokenizer training on the TinyStories dataset.

> 在 TinyStories 数据集上训练 BPE 分词器。

2. Trained tokenizer encoding on the dataset to convert it into a sequence of integer IDs.

> 使用训练好的分词器对数据集进行编码，将其转换为整数 ID 序列。

3. Transformer LM training on the TinyStories dataset.

> 在 TinyStories 数据集上训练 Transformer 语言模型。

4. Sample generation and evaluation of perplexity using the trained Transformer LM.

> 使用训练好的 Transformer LM 进行样本生成和困惑度评估。

5. Model training on OpenWebText, and submit your attained perplexities to a leaderboard.

> 在 OpenWebText 上训练模型，并将你达到的困惑度提交到排行榜。

What you can use

你可以使用的内容

We expect you to build each component from scratch. In particular, you may not use any definitions from `torch.nn`, `torch.nn.functional`, or `torch.optim` except for the following:

- `torch.nn.Parameter`
- Container classes in `torch.nn` (e.g., Module, ModuleList, Sequential, etc.).
- The `torch.optim.Optimizer` base class

> 我们期望你从头构建每个组件。特别地，你不能使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义，但以下内容除外：
>
> - `torch.nn.Parameter`
> - `torch.nn` 中的容器类（例如 Module、ModuleList、Sequential 等）。
> - `torch.optim.Optimizer` 基类

You may use any other PyTorch definitions. If you would like to use a function or class and are not sure whether it is permitted, feel free to ask on Slack. When in doubt, consider if using it compromises the “from-scratch” ethos of the assignment.

> 你可以使用任何其他 PyTorch 定义。如果你想使用某个函数或类但不确定是否允许，请随时在 Slack 上提问。当有疑问时，请考虑使用它是否会损害作业的”从头构建”精神。

Statement on AI tools

关于 AI 工具的声明

AI can solve many parts of the assignments fully autonomously. This makes it harder to deeply engage with, and learn from, the course material.

> AI 可以完全自主地解决作业的许多部分。这使得深入学习课程材料变得更加困难。

The use of AI tools is permitted for answering high-level conceptual questions, or for providing low-level programming documentation like function signatures and library APIs. However, AI tools are not permitted for implementing any part of any assignment. This includes both coding agents (e.g., Cursor Agents, Codex, Claude Code) and AI autocomplete (e.g., Cursor Tab, GitHub Copilot). When using an AI agent, make sure it uses the AGENTS.md file provided. The prompt should also be included when using chatbots.

> 允许使用 AI 工具回答高层次概念性问题，或提供低层次编程文档，如函数签名和库 API。但是，不允许使用 AI 工具来实现作业的任何部分。这包括编码代理（例如 Cursor Agents、Codex、Claude Code）和 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。当使用 AI 代理时，请确保使用提供的 AGENTS.md 文件。使用聊天机器人时也应包含提示词。

We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub Copilot) in your IDE when completing assignments (though non-AI autocomplete, e.g., autocompleting function names is totally fine). Previous students have highlighted that disabling AI autocomplete made it easier to engage deeply with the material.

> 我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动补全（例如 Cursor Tab、GitHub Copilot）（但非 AI 自动补全，例如自动补全函数名是完全可以的）。之前的同学们强调，禁用 AI 自动补全使得深入理解材料变得更容易。

For the full AI Policy, see this document.

> 完整的 AI 政策请参见此文档。

What the code looks like

代码结构

The assignment code, as well as this writeup, are available on GitHub at:

> 作业代码以及本文档可在 GitHub 上获取：

github.com/stanford-cs336/assignment1-basics

Please git clone the repository. If there are any updates, we will notify you so you can git pull to get the latest.

> 请 git clone 该仓库。如果有任何更新，我们会通知你，以便你 git pull 获取最新版本。

1. `cs336_basics/*`: This is where you write your code. Note that there's no code in here–you can do whatever you want from scratch!

> `cs336_basics/*`：这是你编写代码的地方。注意这里没有任何代码——你可以从头开始做任何你想做的事情！

2. `adapters.py`: There is a set of functionality that your code must have. For each piece of functionality (e.g., scaled dot product attention), fill out its implementation (e.g., `run_scaled_dot_product_attention`) by simply invoking your code. Note: your changes to adapters.py should not contain any substantive logic; this is glue code.

> `adapters.py`：你的代码必须实现一组功能。对于每个功能（例如缩放点积注意力），通过简单调用你的代码来填充其实现（例如 `run_scaled_dot_product_attention`）。注意：你对 adapters.py 的修改不应包含任何实质性逻辑；这是胶水代码。

3. `test_*.py`: This contains all the tests that you must pass (e.g., `test_scaled_dot_product_attention`), which will invoke the hooks defined in adapters.py. Don't edit the test files.

> `test_*.py`：这包含你必须通过的所有测试（例如 `test_scaled_dot_product_attention`），这些测试将调用 adapters.py 中定义的钩子。不要编辑测试文件。

How to submit

如何提交

In order to submit, run `make_submission.sh` to construct a submission zip file. Make sure to add any additional files to the list of exclusions in the script if you have large data files or checkpoints you don't want to include in your submission zip.

> 为了提交，运行 `make_submission.sh` 来构建提交的 zip 文件。如果你有不想包含在提交 zip 中的大型数据文件或检查点，请确保在脚本的排除列表中添加这些文件。

You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written.

> 你将向 Gradescope 提交以下文件：
>
> - `writeup.pdf`：回答所有书面问题。请排版你的回答。
>
> - `code.zip`：包含你编写的所有代码。

To submit to the leaderboard, submit a PR to:

> 要提交到排行榜，请向以下仓库提交 PR：

github.com/stanford-cs336/assignment1-basics-leaderboard

See the README.md in the leaderboard repository for detailed submission instructions.

有关详细的提交说明，请参阅排行榜仓库中的 README.md。

Where to get datasets

在哪里获取数据集

This assignment will use two pre-processed datasets: TinyStories [R. Eldan et al., 2023] and OpenWebText [A. Gokaslan et al., 2019]. Both datasets are single, large plaintext files.

本作业将使用两个预处理数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。两个数据集都是单一的大型纯文本文件。

If you are doing the assignment with the class, you can find instructions for downloading the dataset in the compute guide.

如果你是与班级一起完成作业，你可以在计算指南中找到下载数据集的说明。

If you are following along at home, you can download these files with the commands inside the README.md.

如果你是在家中自行学习，你可以使用 README.md 中的命令下载这些文件。

Low-Resource Tip: Init

低资源提示：入门

Throughout the course's assignment handouts, we will give advice for working through parts of the assignment with fewer or no GPU resources. For example, we will sometimes suggest downscaling your dataset or model size, or explain how to run training code on a Mac integrated GPU or CPU. You'll find these "low-resource tips" in a blue box (like this one). Even if you are an enrolled Stanford student with access to the course machines, these tips may help you iterate faster and save time, so we recommend reading them!

在整个课程的作业讲义中，我们会为 GPU 资源较少或没有 GPU 资源的情况下完成部分作业提供建议。例如，我们有时会建议缩小数据集或模型规模，或解释如何在 Mac 集成 GPU 或 CPU 上运行训练代码。你会在蓝色方框中找到这些”低资源提示”（就像这个）。即使你是注册的斯坦福学生并可以使用课程机器，这些提示也可能帮助你更快地迭代并节省时间，所以我们建议阅读它们！

Low-Resource Tip: Assignment 1 on Apple Silicon or CPU

低资源提示：在 Apple Silicon 或 CPU 上完成作业 1

With the staff solution code, we can train an LM to generate reasonably fluent text on an Apple M4 Max chip with 36 GB RAM, in under 5 minutes on Metal GPU (MPS) and about 30 minutes using the CPU. If these words don't mean much to you, don't worry! Just know that if you have a reasonably

> up-to-date laptop and your implementation is correct and efficient, you will be able to train a small LM that generates simple children's stories with decent fluency.
>
> 使用教学团队的参考代码，我们可以在配备 36 GB RAM 的 Apple M4 Max 芯片上训练一个 LM 来生成相当流利的文本，在 Metal GPU（MPS）上不到 5 分钟，使用 CPU 约 30 分钟。如果这些话对你来说意义不大，不用担心！只需要知道，如果你有一台配置较新的笔记本电脑，并且你的实现正确且高效，你将能够训练一个小型 LM，生成具有不错流利度的简单儿童故事。
>
> Later in the assignment, we will explain what changes to make if you are on CPU or MPS.
>
> 在作业的后面部分，我们将解释如果你在 CPU 或 MPS 上运行需要做哪些更改。

---

2 Byte-Pair Encoding (BPE) Tokenizer

字节对编码（BPE）分词器

In the first part of the assignment, we will train and implement a byte-level byte-pair encoding (BPE) tokenizer [R. Sennrich et al., 2016; C. Wang et al., 2019]. In particular, we will represent arbitrary (Unicode) strings as a sequence of bytes and train our BPE tokenizer on this byte sequence. Later, we will use this tokenizer to encode text (a string) into tokens (a sequence of integers) for language modeling.

> 在作业的第一部分，我们将训练并实现一个字节级字节对编码（BPE）分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。特别地，我们将把任意（Unicode）字符串表示为字节序列，并在此字节序列上训练我们的 BPE 分词器。之后，我们将使用该分词器将文本（字符串）编码为 token（整数序列），用于语言建模。

2.1 The Unicode Standard

Unicode 标准

Unicode is a text encoding standard that maps characters to integer code points. As of Unicode 17.0 (released in September 2025), the standard defines 159,801 characters across 172 scripts. For example, the character "s" has the code point 115 (typically notated as U+0073, where U+ is a conventional prefix and 0073 is 115 in hexadecimal), and the character "🐮" has the code point 29275. In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

> Unicode 是一种将字符映射到整数码点的文本编码标准。截至 Unicode 17.0（2025 年 9 月发布），该标准定义了跨 172 种文字的 159,801 个字符。例如，字符 "s" 的码点为 115（通常记作 U+0073，其中 U+ 是约定前缀，0073 是 115 的十六进制表示），字符 "🐮" 的码点为 29275。在 Python 中，你可以使用

`ord()` 函数将单个 Unicode 字符转换为其整数表示。`chr()` 函数将整数 Unicode 码点转换为包含相应字符的字符串。

```
>>> ord('🐮')
29275
>>> chr(29275)
'🐮'
```

Problem (unicode1): Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?

Deliverable: A one-sentence response.

(b) How does this character's string representation (`__repr__()`) differ from its printed representation?

Deliverable: A one-sentence response.

(c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

`python >>> chr(0) >>> print(chr(0)) >>> "this is a test" + chr(0) + "string" >>> print("this is a test" + chr(0) + "string")`

Deliverable: A one-sentence response.

问题（unicode1）：理解 Unicode（1 分）

(a) `chr(0)` 返回什么 Unicode 字符？

交付物：一句话回答。

(b) 这个字符的字符串表示（`__repr__()`）与其打印表示有何不同？

交付物：一句话回答。

(c) 当这个字符出现在文本中时会发生什么？在你的 Python 解释器中尝试以下代码可能会有所帮助，看看是否符合你的预期：

交付物：一句话回答。

2.2 Unicode Encodings

Unicode 编码

While the Unicode standard defines a mapping from characters to code points (integers), it's impractical to train tokenizers directly on Unicode code points, since the vocabulary would be prohibitively large (around 150K items) and sparse (since many characters are quite rare). Instead, we'll use a Unicode encoding, which converts a Unicode character into a sequence of bytes. The Unicode standard itself defines three encodings: UTF-8, UTF-16, and UTF-32, with UTF-8 being the dominant encoding for the Internet (more than 98% of all webpages).

> 虽然 Unicode 标准定义了从字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器是不切实际的，因为词汇表会非常大（约 15 万项）且稀疏（因为许多字符非常罕见）。相反，我们将使用 Unicode 编码，它将 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上的主导编码（超过 98%的网页）。

To encode a Unicode string into UTF-8, we can use the `encode()` function in Python. To access the underlying byte values for a Python bytes object, we can iterate over it (e.g., call `list()`). Finally, we can use the `decode()` function to decode a UTF-8 byte string into a Unicode string.

> 要将 Unicode 字符串编码为 UTF-8，我们可以使用 Python 中的 `encode()` 函数。要访问 Python bytes 对象的底层字节值，我们可以遍历它（例如调用 `list()`）。最后，我们可以使用 `decode()` 函数将 UTF-8 字节字符串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

By converting our Unicode code points into a sequence of bytes (e.g., via the UTF-8 encoding), we are essentially taking a sequence of code points (21-bit integers with 159,801 valid values) and transforming it into a sequence of byte values (integers in the range 0 to 255). The 256-length byte vocabulary is much more manageable to deal with. When using byte-level tokenization, we do not need to worry about out-of-vocabulary tokens, since we know that any input text can be expressed as a sequence of integers from 0 to 255.

> 通过将 Unicode 码点转换为字节序列（例如通过 UTF-8 编码），我们本质上是将码点序列（具有 159,801 个有效值的 21 位整数）转换为字节值序列（范围在 0 到 255 的整数）。256 大小的字节词汇表更容易处

理。当使用字节级分词时，我们不需要担心词汇表外（out-of-vocabulary）token，因为我们知道任何输入文本都可以表示为从 0 到 255 的整数序列。

Problem (unicode2): Unicode Encodings (3 points)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Deliverable: A one-to-two sentence response.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
python     def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
return "".join([bytes([b]).decode("utf-8") for b in bytestring])
```

Deliverable: An example input byte string for which `decode_utf8_bytes_to_str_wrong` produces incorrect output, with a one-sentence explanation of why the function is incorrect.

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

Deliverable: An example, with a one-sentence explanation.

问题（unicode2）：Unicode 编码（3 分）

(a) 为什么我们更倾向于在 UTF-8 编码的字节上训练分词器，而不是 UTF-16 或 UTF-32？比较这些编码对各种输入字符串的输出可能会有所帮助。

交付物：一到两句话回答。

(b) 考虑以下（不正确的）函数，它旨在将 UTF-8 字节字符串解码为 Unicode 字符串。为什么这个函数是不正确的？提供一个会产生错误结果的输入字节字符串示例。

交付物：一个 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节字符串示例，以及一句话解释为什么该函数不正确。

(c) 给出一个不能解码为任何 Unicode 字符的两字节序列。

交付物：一个示例，以及一句话解释。

2.3 Subword Tokenization

子词分词

While byte-level tokenization can alleviate the out-of-vocabulary issues faced by word-level tokenizers, tokenizing text into bytes results in extremely long input sequences. This slows down model training, since a sentence with 10 words might only be 10 tokens long in a word-level language model, but could be 50 or more tokens long in a character-level model (depending on the length of the words). Processing these longer sequences requires more computation at each step of the model. Furthermore, language modeling on byte sequences is difficult because the longer input sequences create long-term dependencies in the data.

> 虽然字节级分词可以缓解词级分词器面临的词汇表外问题，但将文本分词为字节会导致极长的输入序列。这会减慢模型训练，因为一个包含 10 个单词的句子在词级语言模型中可能只有 10 个 token，但在字符级模型中可能有 50 个或更多 token（取决于单词的长度）。处理这些更长的序列需要在模型的每一步进行更多计算。此外，在字节序列上进行语言建模是困难的，因为更长的输入序列会在数据中产生长期依赖关系。

Subword tokenization is a midpoint between word-level tokenizers and byte-level tokenizers. Note that a byte-level tokenizer's vocabulary has 256 entries (byte values are 0 to 255). A subword tokenizer trades off a larger vocabulary size for better compression of the input byte sequence. For example, if the byte sequence `b'the'` often occurs in our raw text training data, assigning it an entry in the vocabulary would reduce this 3-token sequence to a single token.

> 子词分词是词级分词器和字节级分词器之间的折中方案。注意，字节级分词器的词汇表有 256 个条目（字节值为 0 到 255）。子词分词器通过更大的词汇表大小来换取更好的输入字节序列压缩。例如，如果字节序列 `b'the'` 经常出现在我们的原始文本训练数据中，在词汇表中为其分配一个条目会将这个 3-token 序列减少为单个 token。

How do we select these subword units to add to our vocabulary? R. Sennrich et al. [3] propose to use byte-pair encoding (BPE; P. Gage [5]), a compression algorithm that iteratively replaces ("merges") the most frequent pair of bytes with a single, new unused index. Note that this algorithm adds subword tokens to our vocabulary to maximize the compression of our input sequences–if a word occurs in our input text enough times, it'll be represented as a single subword unit.

> 我们如何选择这些子词单元来添加到词汇表中？R. Sennrich 等人 [3] 提出使用字节对编码（BPE；P. Gage [5]），这是一种压缩算法，通过迭代地将最频繁的字节对替换（“合并”）为一个新的、未使用的索引。注意，该算法将子词 token 添加到我们的词汇表中，以最大化输入序列的压缩——如果一个词在输入文本中出现足够多次，它将被表示为单个子词单元。

Subword tokenizers with vocabularies constructed via BPE are often called BPE tokenizers. In this assignment, we'll implement a byte-level BPE tokenizer, where the vocabulary items are bytes or merged sequences of bytes, which give us the best of both worlds in terms of out-of-vocabulary handling and manageable input sequence lengths. The process of constructing the BPE tokenizer vocabulary is known as "training" the BPE tokenizer.

> 通过 BPE 构建词汇表的子词分词器通常被称为 BPE 分词器。在本作业中，我们将实现一个字节级 BPE 分词器，其中词汇表项是字节或合并的字节序列，这让我们在词汇表外处理和可管理的输入序列长度方面兼得两者的优势。构建 BPE 分词器词汇表的过程被称为”训练”BPE 分词器。

2.4 BPE Tokenizer Training

BPE 分词器训练

The BPE tokenizer training procedure consists of three main steps.

> BPE 分词器训练过程包括三个主要步骤。

Vocabulary initialization

词汇表初始化

The tokenizer vocabulary is a one-to-one mapping from bytestring token to integer ID. Since we're training a byte-level BPE tokenizer, our initial vocabulary is simply the set of all bytes. Since there are 256 possible byte values, our initial vocabulary is of size 256.

> 分词器词汇表是从字节串 token 到整数 ID 的一对一映射。由于我们正在训练字节级 BPE 分词器，我们的初始词汇表就是所有字节的集合。由于有 256 个可能的字节值，我们的初始词汇表大小为 256。

Pre-tokenization

预分词

Once you have a vocabulary, you could, in principle, count how often bytes occur next to each other in your text and begin merging them starting with the most frequent pair of bytes. However, this is quite computationally expensive, since we'd have to take a full pass over the corpus each time we merge. In addition, directly merging bytes across the corpus may result in tokens that differ only in punctuation (e.g., dog! vs. dog.). These tokens would get completely different token IDs, even though they are likely to have high semantic similarity (since they differ only in punctuation).

> 一旦有了词汇表，原则上你可以统计文本中字节相邻出现的频率，并从最频繁的字节对开始合并它们。然而，这在计算上是相当昂贵的，因为每次合并时我们都必须遍历整个语料库。此外，在语料库中直接合并字节可能会导致仅在标点符号上有所不同的 token（例如 dog! 与 dog.）。这些 token 将获得完全不同的 token ID，即使它们很可能具有很高的语义相似性（因为它们仅在标点符号上不同）。

To avoid this, we pre-tokenize the corpus. You can think of this as a coarse-grained tokenization over the corpus that helps us count how often pairs of characters appear. For example, the word 'text' might be a pre-token that appears 10 times. In this case, when we count how often the characters 't' and 'e' appear next to each other, we will see that the word 'text' has 't' and 'e' adjacent and we can increment their count by 10 instead of looking through the corpus. Since we're training a byte-level BPE model, each pre-token is represented as a sequence of UTF-8 bytes.

> 为了避免这种情况，我们对语料库进行预分词（pre-tokenization）。你可以将其视为对语料库进行粗粒度分词，帮助我们统计字符对出现的频率。例如，单词 'text' 可能是一个出现 10 次的预分词。在这种情况下，当我们统计字符 't' 和 'e' 相邻出现的频率时，我们会看到单词 'text' 中有 't' 和 'e' 相邻，我们可以将它们的计数增加 10，而无需遍历整个语料库。由于我们正在训练字节级 BPE 模型，每个预分词表示为 UTF-8 字节序列。

The original BPE implementation of R. Sennrich et al. [3] pre-tokenizes by simply splitting on whitespace (i.e., `s.split(" ")`). This method is still found in tokenizers based on SentencePiece (for instance the Llama 1 and 2 tokenizer).

> R. Sennrich 等人 [3] 的原始 BPE 实现通过简单地按空白字符分割（即 `s.split(" ")`）进行预分词。这种方法仍然存在于基于 SentencePiece 的分词器中（例如 Llama 1 和 2 的分词器）。

Most modern tokenizers use a regex-based pre-tokenizer, a practice from GPT-2; A. Radford et al. [6]. We'll use a slightly prettier form of the original regex, fetched from github.com/openai/tiktoken/pull/234/files:

> 大多数现代分词器使用基于正则表达式的预分词器，这是 GPT-2 的做法；A. Radford 等人 [6]。我们将使用原始正则表达式的一个稍微更整洁的形式，取自 github.com/openai/tiktoken/pull/234/files：

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

It may be useful to interactively split some text with this pre-tokenizer to get a better sense of its behavior:

> 交互式地用这个预分词器分割一些文本，以更好地理解其行为，可能会有所帮助：

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

When using it in your code, however, you should use `re.finditer` to avoid storing the pre-tokenized words as you construct your mapping from pre-tokens to their counts.

> 然而，在你的代码中使用它时，你应该使用 `re.finditer`，以避免在构建从预分词到其计数的映射时存储预分词后的单词。

Compute BPE merges

计算 BPE 合并

Now that we've converted our input text into pre-tokens and represented each pre-token as a sequence of UTF-8 bytes, we can compute the BPE merges (i.e., train the BPE tokenizer). At a high level, the BPE algorithm iteratively counts every pair of bytes and identifies the pair with the highest frequency ("A", "B"). Every occurrence of this most frequent pair ("A", "B") is then merged, i.e., replaced with a new token "AB". This new merged token is added to our vocabulary; as a result, the final vocabulary after BPE training is the size of the initial vocabulary (256 in our case), plus the number of BPE merge operations performed during training. For efficiency during BPE training, we do not consider pairs that cross pre-token boundaries. When computing merges, deterministically break ties in pair frequency by preferring the lexicographically greater pair. For example, if the pairs ("A", "B"), ("A", "C"), ("B", "ZZ"), and ("BA", "A") all have the highest frequency, we'd merge ("BA", "A"):

> 现在我们已经将输入文本转换为预分词，并将每个预分词表示为 UTF-8 字节序列，我们可以计算 BPE 合并（即训练 BPE 分词器）。从高层次来看，BPE 算法迭代地统计每对字节的频率，并识别出频率最高的字节对（“A”，“B”）。然后，这个最频繁对（“A”，“B”）的每次出现都被合并，即替换为新的 token “AB”。这个新的合并 token 被添加到我们的词汇表中；因此，BPE 训练后的最终词汇表大小等于初始词汇表大小（在我们的情况下为 256）加上训练期间执行的 BPE 合并操作次数。为了在 BPE 训练期间提高效率，我们不考虑跨越预分词边界的字节对。当计算合并时，通过在频率相同的情况下优先选择字典序更大的字节对来确定性地打破平局。例如，如果字节对（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）都具有最高频率，我们将合并（“BA”，“A”）：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

Special tokens

特殊 token

Often, some strings (e.g., `<|endoftext|>`) are used to encode metadata (e.g., boundaries between documents). When encoding text, it's often desirable to treat some strings as "special tokens" that should never be split into multiple tokens (i.e., will always be preserved as a single token). For example, the end-of-sequence string `<|endoftext|>` should always be preserved as a single token (i.e., a single integer ID), so we know when to stop generating from the language model. These special tokens must be added to the vocabulary, so they have a corresponding fixed token ID.

> 通常，某些字符串（例如 `<|endoftext|>`）用于编码元数据（例如文档之间的边界）。在编码文本时，通常希望将某些字符串视为”特殊 token”，它们永远不应被分割成多个 token（即始终保留为单个 token）。例如，序列结束字符串 `<|endoftext|>` 应始终保留为单个 token（即单个整数 ID），这样我们就知道何时停止从语言模型生成。这些特殊 token 必须添加到词汇表中，以便它们具有相应的固定 token ID。

Algorithm 1 of R. Sennrich et al. [3] contains an inefficient implementation of BPE tokenizer training (essentially following the steps that we outlined above). As a first exercise, it may be useful to implement and test this function to check your understanding.

R. Sennrich 等人 [3] 的算法 1 包含了一个低效的 BPE 分词器训练实现（基本上遵循我们上面概述的步骤）。作为第一个练习，实现并测试这个函数以检查你的理解可能会很有用。

Example (bpe_example): BPE training example

示例（bpe_example）：BPE 训练示例

Here is a stylized example from R. Sennrich et al. [3]. Consider a corpus consisting of the following text

以下是来自 R. Sennrich 等人 [3] 的一个风格化示例。考虑由以下文本组成的语料库

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

and the vocabulary has a special token `<|endoftext|>`.

并且词汇表有一个特殊 token `<|endoftext|>`。

Vocabulary

词汇表

We initialize our vocabulary with our special token `<|endoftext|>` and the 256 byte values.

我们用特殊 token `<|endoftext|>` 和 256 个字节值初始化词汇表。

Pre-tokenization

预分词

For simplicity and to focus on the merge procedure, we assume in this example that pre-tokenization simply splits on whitespace. When we pre-tokenize and count, we end up with the frequency table.

为了简单起见并聚焦于合并过程，在本示例中我们假设预分词只是按空白字符分割。当我们进行预分词和计数时，我们得到以下频率表。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

It is convenient to represent this as a `dict[tuple[bytes, ...], int]`, e.g. `{(l,o,w): 5, ...}`. Note that even a single byte is a bytes object in Python. There is no byte type in Python to represent a single byte, just as there is no char type in Python to represent a single character.

将其表示为 `dict[tuple[bytes, ...], int]` 很方便，例如 `{(l,o,w): 5, ...}`。注意，即使单个字节在 Python 中也是 bytes 对象。Python 中没有表示单个字节的 byte 类型，就像 Python 中没有表示单个字符的 char 类型一样。

Merges

合并

We first look at every successive pair of bytes and sum the frequency of the words where they appear {lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}. The pairs ('e', 's') and ('s', 't') are tied, so we take the lexicographically greater pair, ('s', 't'). We would then merge the pre-tokens so that we end up with `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}`.

我们首先查看每个连续的字节对，并求它们出现的词的频率之和 {lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}。字节对（'e'，'s'）和（'s'，'t'）频率相同，所以我们取字典序更大的对（'s'，'t'）。然后我们合并预分词，得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}`。

In the second round, we see that (e, st) is the most common pair (with a count of 9) and we would merge into `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`. Continuing this, the sequence of merges we get in the end will be ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r'].

在第二轮中，我们看到 (e, st) 是最常见的对（计数为 9），我们将合并为 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`。继续这个过程，我们最终得到的合并序列将是 ['s t'，'e st'，'o w'，'l ow'，'w est'，'n e'，'ne west'，'w i'，'wi d'，'wid est'，'low e'，'lowe r']。

If we take 6 merges, we have ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e'] and our vocabulary elements would be `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`.

如果我们进行 6 次合并，我们得到 ['s t'，'e st'，'o w'，'l ow'，'w est'，'n e']，词汇表元素将是 `[<|endoftext|>, [...256 字节字符], st, est, ow, low, west, ne]`。

With this vocabulary and set of merges, the word `newest` would tokenize as `[ne, west]`.

使用这个词汇表和合并集合，单词 `newest` 将被分词为 `[ne, west]`。

2.5 Experimenting with BPE Tokenizer Training

BPE 分词器训练实验

Let's train a byte-level BPE tokenizer on the TinyStories dataset. Instructions to find / download the dataset can be found in Section 1. Before you start, we recommend taking a look at the TinyStories dataset to get a sense of what's in the data.

> 让我们在 TinyStories 数据集上训练一个字节级 BPE 分词器。查找/下载数据集的说明可在第 1 节中找到。
>
> 在开始之前，我们建议先看一下 TinyStories 数据集，以了解数据中的内容。

Parallelizing pre-tokenization

并行化预分词

You will find that a major bottleneck is the pre-tokenization step. You can speed up pre-tokenization by parallelizing your code with the built-in library `multiprocessing`. Concretely, we recommend that in parallel implementations of pre-tokenization, you chunk the corpus while ensuring your chunk boundaries occur at the beginning of a special token. You are free to use the starter code at the following link verbatim to obtain chunk boundaries, which you can then use to distribute work across your processes:

> 你会发现一个主要瓶颈是预分词步骤。你可以通过使用内置库 `multiprocessing` 并行化代码来加速预分词。具体来说，我们建议在预分词的并行实现中，将语料库分块，同时确保块边界出现在特殊 token 的开始处。你可以逐字使用以下链接中的起始代码来获取块边界，然后将其用于在进程之间分配工作：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

This chunking will always be valid, since we never want to merge across document boundaries. For the purposes of the assignment, you can always split in this way. Don't worry about the edge case of receiving a very large corpus that does not contain `<|endoftext|>`.

> 这种分块方式始终是有效的，因为我们永远不希望跨越文档边界进行合并。对于本作业的目的，你始终可以以这种方式分割。不必担心接收到不包含 `<|endoftext|>` 的非常大的语料库的边缘情况。

Removing special tokens before pre-tokenization

在预分词之前移除特殊 token

Before running pre-tokenization with the regex pattern (using `re.finditer`), you should strip out all special tokens from your corpus (or your chunk, if using a parallel implementation). Make sure that you split on your special tokens, so that no merging can occur across the text they delimit. For example, if you have a corpus (or chunk) like `[Doc 1]<|endoftext|>[Doc 2]`, you should split on the special token `<|endoftext|>`, and pre-tokenize `[Doc 1]` and `[Doc 2]` separately, so that no merging can occur across the document boundary. In other words, special tokens define hard segmentation boundaries during training, but they should not themselves contribute to merge counts. This can be done using `re.split` with `"|".join(special_tokens)` as the delimiter (with careful use of `re.escape` since `|` may occur in the special tokens). The test `test_train_bpe_special_tokens` will test for this.

> 在使用正则表达式模式（使用 `re.finditer`）运行预分词之前，你应该从语料库（或块，如果使用并行实现）中移除所有特殊 token。确保你在特殊 token 处进行分割，以便在它们分隔的文本之间不会发生合并。例如，如果你有一个类似 `[Doc 1]<|endoftext|>[Doc 2]` 的语料库（或块），你应该在特殊 token `<|endoftext|>` 处分割，并分别预分词 `[Doc 1]` 和 `[Doc 2]`，这样就不会跨越文档边界进行合并。换句话说，特殊 token 在训练期间定义了硬分割边界，但它们本身不应该贡献于合并计数。这可以使用 `re.split` 以 `"|".join(special_tokens)` 作为分隔符（谨慎使用 `re.escape`，因为 `|` 可能出现在特殊 token 中）来完成。测试 `test_train_bpe_special_tokens` 将测试这一点。

Optimizing the merging step

优化合并步骤

The naïve implementation of BPE training in the stylized example above is slow because for every merge, it iterates over all byte pairs to identify the most frequent pair. However, the only pair counts that change after each merge are those that overlap with the merged pair. Thus, BPE training speed can be improved by indexing the counts of all pairs and incrementally updating these counts, rather than explicitly iterating over each pair of bytes to count pair frequencies. You can get significant speedups with this caching procedure, though we note that the merging part of BPE training is not parallelizable in Python.

> 上述风格化示例中 BPE 训练的朴素实现很慢，因为对于每次合并，它都会遍历所有字节对来识别最频繁的对。然而，每次合并后唯一改变的字节对计数是那些与合并对重叠的对。因此，可以通过索引所有对的计数并增量更新这些计数来提高 BPE 训练速度，而不是显式遍历每个字节对来计算对频率。你可以通过这种缓存过程获得显著的加速，尽管我们注意到 BPE 训练的合并部分在 Python 中是不可并行化的。

Low-Resource Tip: Profiling

低资源提示：性能分析

You should use profiling tools like `cProfile` or `py-spy` to identify the bottlenecks in your implementation, and focus on optimizing those.

你应该使用像 `cProfile` 或 `py-spy` 这样的性能分析工具来识别实现中的瓶颈，并专注于优化它们。

Low-Resource Tip: "Downscaling"

低资源提示："缩减规模"

Instead of jumping to training your tokenizer on the full TinyStories dataset, we recommend you first train on a small subset of the data: a "debug dataset". For example, you could train your tokenizer on the TinyStories validation set instead, which is 22K documents instead of 2.12M. This illustrates a general strategy of downscaling whenever possible to speed up development: for example, using smaller datasets, smaller model sizes, etc. Choosing the size of the debug dataset or hyperparameter config requires careful consideration: you want your debug set to be large enough to have the same bottlenecks as the full configuration (so that the optimizations you make will generalize), but not so big that it takes forever to run.

与其直接在整个 TinyStories 数据集上训练分词器，我们建议你首先在一个小的数据子集上训练：一个"调试数据集"。例如，你可以在 TinyStories 验证集上训练分词器，该验证集有 22K 篇文档，而不是 2.12M。这展示了一种尽可能缩减规模以加速开发的通用策略：例如，使用更小的数据集、更小的模型规模等。选择调试数据集或超参数配置的大小需要仔细考虑：你希望调试集足够大，以具有与完整配置相同的瓶颈（这样你所做的优化才能泛化），但又不能太大以至于运行时间过长。

Problem (train_bpe): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

Input

- `input_path: str` — Path to a text file with BPE tokenizer training data.
- `vocab_size: int` — A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).
- `special_tokens: list[str]` — A list of strings to add to the vocabulary. During training, treat them as hard boundaries that prevent merges across their spans, but do not include them when computing merge statistics.

Your BPE training function should return the resulting vocabulary and merges:

Output

- `vocab: dict[int, bytes]` — The tokenizer vocabulary, a mapping from int (token ID in the vocabulary) to bytes (token bytes).
- `merges: list[tuple[bytes, bytes]]` — A list of BPE merges produced from training. Each list item is a tuple of bytes (<token1>, <token2>), representing that <token1> was merged with <token2>. The merges should be ordered by order of creation.

To test your BPE training function against our provided tests, you will first need to implement the test adapter at `adapters.run_train_bpe`. Then, run `uv run pytest tests/test_train_bpe.py`.

Your implementation should be able to pass all tests. Optionally (this could be a large time-investment), you can implement the key parts of your training method using some systems language, for instance C++ (consider cppyy or nanobind) or Rust (using PyO3). If you do this, be aware of which operations require copying vs reading directly from Python memory, and make sure to leave build instructions, or make sure it builds using only pyproject.toml. Also note that the GPT-2 regex is not well-supported in most regex engines and will be too slow in most that do. We have verified that Oniguruma is reasonably fast and supports negative lookahead, but the regex package in Python is, if anything, even faster.

问题（train_bpe）：BPE 分词器训练（15 分）

交付物：编写一个函数，给定输入文本文件的路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应该处理（至少）以下输入参数：

输入

包含 BPE 分词器训练数据的文本文件路径。

一个正整数，定义最终词汇表的最大大小（包括初始字节词汇表、合并产生的词汇表项以及任何特殊 token）。

要添加到词汇表中的字符串列表。在训练期间，将它们视为硬边界，防止跨其范围进行合并，但在计算合并统计时不包括它们。

你的 BPE 训练函数应该返回结果词汇表和合并：

输出

分词器词汇表，从 int（词汇表中的 token ID）到 bytes（token 字节）的映射。

训练产生的 BPE 合并列表。每个列表项是一个 bytes 元组（<token1>, <token2>），表示 <token1> 与 <token2> 合并。合并应按创建顺序排列。

要根据我们提供的测试来测试你的 BPE 训练函数，你首先需要实现 adapters.run_train_bpe 处的测试适配器。然后运行 uv run pytest tests/test_train_bpe.py。你的实现应该能够通过所有测试。可选地（这可能是一个大的时间投入），你可以使用某种系统语言实现训练方法的关键部分，例如 C++（考虑 cppyy 或 nanobind）或 Rust（使用 PyO3）。如果你这样做，要注意哪些操作需要复制，哪些操作可以直接从 Python 内存读取，并确保留下构建说明，或确保仅使用 pyproject.toml 即可构建。还要注意，GPT-2 正则表达式在大多数正则引擎中没有得到很好的支持，并且在大多数支持的引擎中也会太慢。我们已经验证 Oniguruma 相当快并且支持负向前瞻，但 Python 中的 regex 包甚至更快。

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories <|endoftext|> special token to the vocabulary.

Serialize the resulting vocabulary and merges to disk for further inspection. How much time and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≈ 30 minutes (no GPUs), ≈ 30 GB RAM

Hint: You should be able to get under 2 minutes for BPE training using multiprocessing during pre-tokenization and the following two facts:

(a) The `<|endoftext|>` token delimits documents in the data files.

(b) The `<|endoftext|>` token is handled as a special case before the BPE merges are applied.

Deliverable: A one-to-two sentence response.

(b) Profile your code. What part of the tokenizer training process takes the most time?

Deliverable: A one-to-two sentence response.

问题（train_bpe_tinystories）：在 TinyStories 上进行 BPE 训练（2 分）

在 TinyStories 数据集上训练一个字节级 BPE 分词器，使用最大词汇表大小 10,000。确保将 TinyStories 的 `<|endoftext|>` 特殊 token 添加到词汇表中。将结果词汇表和合并序列化到磁盘以供进一步检查。训练花费了多少时间和内存？词汇表中最长的 token 是什么？它有意义吗？

资源需求：约 30 分钟（无 GPU），约 30 GB RAM

提示：你应该能够在预分词期间使用多进程处理以及以下两个事实，将 BPE 训练时间控制在 2 分钟以内：

(a) `<|endoftext|>` token 在数据文件中分隔文档。

(b) `<|endoftext|>` token 在 BPE 合并应用之前作为特殊情况处理。

交付物：一到两句话回答。

(b) 对你的代码进行性能分析。分词器训练过程的哪部分花费的时间最多？

交付物：一到两句话回答。

Next, we'll try training a byte-level BPE tokenizer on the OpenWebText dataset. As before, we recommend taking a look at the dataset to better understand its contents.

接下来，我们将尝试在 OpenWebText 数据集上训练一个字节级 BPE 分词器。和之前一样，我们建议先看一下数据集以更好地理解其内容。

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≈ 12 hours (no GPUs), ≈ 100 GB RAM

Deliverable: A one-to-two sentence response.

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Deliverable: A one-to-two sentence response.

问题（train_bpe_expts_owt）：在 OpenWebText 上进行 BPE 训练（2 分）

在 OpenWebText 数据集上训练一个字节级 BPE 分词器，使用最大词汇表大小 32,000。将结果词汇表和合并序列化到磁盘以供进一步检查。词汇表中最长的 token 是什么？它有意义吗？

资源需求：约 12 小时（无 GPU），约 100 GB RAM

交付物：一到两句话回答。

(b) 比较和对比你在 TinyStories 和 OpenWebText 上训练得到的分词器。

交付物：一到两句话回答。

2.6 BPE Tokenizer: Encoding and Decoding

BPE 分词器：编码和解码

In the previous part of the assignment, we implemented a function to train a BPE tokenizer on input text to obtain a tokenizer vocabulary and a list of BPE merges. Now, we will implement a BPE tokenizer that loads a provided vocabulary and list of merges and uses them to encode and decode text to/from token IDs.

在作业的前一部分，我们实现了一个函数，在输入文本上训练 BPE 分词器，以获得分词器词汇表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器，它加载提供的词汇表和合并列表，并使用它们将文本编码为 token ID 和从 token ID 解码为文本。

2.6.1 Encoding text

编码文本

The process of encoding text by BPE mirrors how we train the BPE vocabulary. There are a few major steps.

通过 BPE 编码文本的过程反映了我们训练 BPE 词汇表的方式。有几个主要步骤。

Step 1: Pre-tokenize. We first pre-tokenize the sequence and represent each pre-token as a sequence of UTF-8 bytes, just as we did in BPE training. We will be merging these bytes within each pre-token into vocabulary elements, handling each pre-token independently (no merges across pre-token boundaries).

步骤 1：预分词。我们首先对序列进行预分词，并将每个预分词表示为 UTF-8 字节序列，就像我们在 BPE 训练中所做的那样。我们将在每个预分词内将这些字节合并为词汇表元素，独立处理每个预分词（不在预分词边界之间进行合并）。

Step 2: Apply the merges. We then take the sequence of vocabulary element merges created during BPE training, and apply it to our pre-tokens in the same order of creation.

步骤 2：应用合并。然后我们获取 BPE 训练期间创建的词汇表元素合并序列，并按相同的创建顺序将其应用到我们的预分词上。

Example (bpe_encoding): BPE encoding example

示例（bpe_encoding）：BPE 编码示例

For example, suppose our input string is `'the cat ate'`, our vocabulary is `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`, and our learned merges are `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]`. First, our pre-tokenizer would split this string into `['the', ' cat', ' ate']`. Then, we'll look at each pre-token and apply the BPE merges.

例如，假设我们的输入字符串是 `'the cat ate'`，词汇表是 `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`，我们学到的合并是 `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]`。首先，我们的预分词器将这个字符串分割为 `['the', ' cat', ' ate']`。然后，我们查看每个预分词并应用 BPE 合并。

The first pre-token `'the'` is initially represented as `[b't', b'h', b'e']`. Looking at our list of merges, we identify the first applicable merge to be `(b't', b'h')`, and use that to transform the pre-token into `[b'th', b'e']`. Then, we go back to the list of merges and identify the next applicable merge to be `(b'th', b'e')`, which transforms the pre-token into `[b'the']`. Finally, looking back at the list of merges, we see that there are no more that apply to the string (since the entire pre-token has been merged into a single token), so we are done applying the BPE merges. The corresponding integer sequence is `[9]`.

第一个预分词 `'the'` 最初表示为 `[b't', b'h', b'e']`。查看我们的合并列表，我们识别出第一个适用的合并是 `(b't', b'h')`，并使用它将预分词转换为 `[b'th', b'e']`。然后，我们回到合并列表，识别出下一个适用的合并是 `(b'th', b'e')`，它将预分词转换为 `[b'the']`。最后，回顾合并列表，我们看到没有更多适用于该字符串的合并（因为整个预分词已合并为单个 token），因此我们完成了 BPE 合并的应用。相应的整数序列是 `[9]`。

Repeating this process for the remaining pre-tokens, we see that the pre-token `' cat'` is represented as `[b' c', b'a', b't']` after applying the BPE merges, which becomes the integer sequence `[7, 1, 5]`. The final pre-token `' ate'` is `[b' at', b'e']` after applying the BPE merges, which becomes the integer sequence `[10, 3]`. Thus, the final result of encoding our input string is `[9, 7, 1, 5, 10, 3]`.

对剩余的预分词重复此过程，我们看到预分词 `' cat'` 在应用 BPE 合并后表示为 `[b' c', b'a', b't']`，对应的整数序列为 `[7, 1, 5]`。最后的预分词 `' ate'` 在应用 BPE 合并后为 `[b' at', b'e']`，对应的整数序列为 `[10, 3]`。因此，编码我们输入字符串的最终结果是 `[9, 7, 1, 5, 10, 3]`。

Special tokens

特殊 token

Your tokenizer should be able to properly handle user-defined special tokens when encoding text (provided when constructing the tokenizer).

你的分词器在编码文本时应该能够正确处理用户定义的特殊 token（在构建分词器时提供）。

Memory considerations

内存考虑

Suppose we want to tokenize a large text file that we cannot fit in memory. To efficiently tokenize this large file (or any other stream of data), we need to break it up into manageable chunks and process each chunk in turn, so that the memory complexity is constant as opposed to linear in the size of the text. In doing so, we need to make sure that a token doesn't cross chunk boundaries, else we'll get a different tokenization than the naïve method of tokenizing the entire sequence in-memory.

假设我们想要对一个无法放入内存的大型文本文件进行分词。为了高效地对这个大文件（或任何其他数据流）进行分词，我们需要将其分解为可管理的块并依次处理每个块，这样内存复杂度是常数的，而不是与文本大小成线性关系。在此过程中，我们需要确保 token 不会跨越块边界，否则我们会得到与将整个序列加载到内存中分词的朴素方法不同的分词结果。

2.6.2 Decoding text

解码文本

To decode a sequence of integer token IDs back to raw text, we can simply look up each ID's corresponding entries in the vocabulary (a byte sequence), concatenate them together, and then decode the bytes to a Unicode string. Note that input IDs are not guaranteed to map to valid Unicode strings (since a user could input any sequence of integer IDs). In the case that the input token IDs do not produce a valid Unicode string, you should replace the malformed bytes with the official Unicode replacement character U+FFFD. The `errors` argument of `bytes.decode` controls how Unicode decoding errors are handled, and using `errors='replace'` will automatically replace malformed data with the replacement marker.

要将整数 token ID 序列解码回原始文本，我们可以简单地查找每个 ID 在词汇表中的对应条目（字节序列），将它们连接在一起，然后将字节解码为 Unicode 字符串。注意，输入的 ID 不保证能映射到有效的 Unicode 字符串（因为用户可以输入任何整数 ID 序列）。在输入 token ID 不产生有效 Unicode 字符串的

情况下，你应该用官方的 Unicode 替换字符 U+FFFD 替换格式错误的字节。`bytes.decode` 的 `errors` 参数控制如何处理 Unicode 解码错误，使用 `errors='replace'` 将自动用替换标记替换格式错误的数据。

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there). We recommend the following interface:

- `__init__(self, vocab, merges, special_tokens=None)` — Construct a tokenizer from a given vocabulary, list of merges, and (optionally) a list of special tokens.
  - `vocab: dict[int, bytes]`
  - `merges: list[tuple[bytes, bytes]]`
  - `special_tokens: list[str] | None = None`
- `from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)` — Class method that constructs and returns a Tokenizer from a serialized vocabulary and list of merges (in the same format that your BPE training code output) and (optionally) a list of special tokens.
  - `vocab_filepath: str`
  - `merges_filepath: str`
  - `special_tokens: list[str] | None = None`
- `encode(self, text: str) -> list[int]` — Encode an input text into a sequence of token IDs.
- `encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]` — Given an iterable of strings (e.g., a Python file handle), return a generator that lazily yields token IDs. This is required for memory-efficient tokenization of large files that we cannot directly load into memory.
- `decode(self, ids: list[int]) -> str` — Decode a sequence of token IDs into text.

To test your Tokenizer against our provided tests, you will first need to implement the test adapter at `adapters.get_tokenizer`. Then, run `uv run pytest tests/test_tokenizer.py`. Your implementation should be able to pass all tests.

问题（tokenizer）：实现分词器（15 分）

交付物：实现一个 `Tokenizer` 类，给定词汇表和合并列表，将文本编码为整数 ID 并将整数 ID 解码为文本。你的分词器还应支持用户提供的特殊 token（如果它们不在词汇表中，则将其追加到词汇表中）。我们建议以下接口：

- 从给定的词汇表、合并列表和（可选地）特殊 token 列表构造分词器。
- 类方法，从序列化的词汇表和合并列表（与你的 BPE 训练代码输出格式相同）以及（可选地）特殊 token 列表构造并返回 Tokenizer。
- 将输入文本编码为 token ID 序列。

- 给定一个字符串的可迭代对象（例如 Python 文件句柄），返回一个惰性生成 token ID 的生成器。这对于无法直接加载到内存中的大型文件的内存高效分词是必需的。
- 将 token ID 序列解码为文本。

要根据我们提供的测试来测试你的 Tokenizer，你首先需要实现 `adapters.get_tokenizer` 处的测试适配器。然后运行 `uv run pytest tests/test_tokenizer.py`。你的实现应该能够通过所有测试。

2.7 Experiments

实验

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Deliverable: A one-to-two sentence response.

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Deliverable: A one-to-two sentence response.

(c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Deliverable: A one-to-two sentence response.

(d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Deliverable: A one-to-two sentence response.

问题（tokenizer_experiments）：分词器实验（4 分）

从 TinyStories 和 OpenWebText 中各抽样 10 篇文档。使用你之前训练的 TinyStories 和 OpenWebText 分词器（词汇表大小分别为 10K 和 32K），将这些抽样文档编码为整数 ID。每个分词器的压缩比（字节/token）是多少？

交付物：一到两句话回答。

(b) 如果你用 TinyStories 分词器对 OpenWebText 样本进行分词会发生什么？比较压缩比和/或定性描述发生了什么。

交付物：一到两句话回答。

(c) 估算你的分词器的吞吐量（例如以字节/秒为单位）。分词 Pile 数据集（825GB 文本）需要多长时间？

交付物：一到两句话回答。

(d) 使用你的 TinyStories 和 OpenWebText 分词器，将相应的训练和开发数据集编码为整数 token ID 序列。我们稍后将使用它来训练我们的语言模型。我们建议将 token ID 序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是一个合适的选择？

交付物：一到两句话回答。

3 Transformer Language Model Architecture

Transformer 语言模型架构

A language model takes as input a batched sequence of integer token IDs (i.e., a `torch.Tensor` of shape `(batch_size, sequence_length)`), and returns a batched normalized probability distribution over the vocabulary (i.e., a PyTorch tensor of shape `(batch_size, sequence_length, vocab_size)`), where the predicted distribution is over the next word for each input token. When training the language model, we use these next-word predictions to calculate the cross-entropy loss between the actual next word and the predicted next word. When generating text from the language model during inference, we take the predicted next-word distribution from the final time step (i.e., the last item in the sequence) to generate the next token in the sequence (e.g., by taking the token with the highest probability or sampling from the distribution), add the generated token to the input sequence, and repeat.

语言模型接收一批整数词元 ID 序列作为输入（即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`），并返回词表上的一批归一化概率分布（即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch 张量）；对每个输入词元而言，该分布预测的是下一个词。训练语言模型时，我们用这些下一词预测计算真实下一词与预测下一词之间的交叉熵损失。推理生成文本时，我们取最后一个时间步（即序列中的最后一项）的下一词预测分布来生成序列中的下一个词元（例如选择概率最高的词元或从分布中采样），把生成的词元加入输入序列，然后重复这一过程。

In this part of the assignment, you will build this Transformer language model from scratch. We will begin with a high-level description of the model before progressively detailing the individual components.

在本作业的这一部分，你将从头构建这个 Transformer 语言模型。我们会先从整体层面介绍模型，然后逐步详述各个组件。

3.1 Transformer LM

Transformer 语言模型

Given a sequence of token IDs, the Transformer language model uses an input embedding to convert token IDs to dense vectors, passes the embedded tokens through `num_layers` Transformer blocks, and then applies a learned linear projection (the "output embedding" or "LM head") to produce the predicted next-token logits. See Figure 1 for a schematic representation.

给定一个词元 ID 序列，Transformer 语言模型先用输入嵌入把词元 ID 转换为稠密向量，再让嵌入后的词元经过 `num_layers` 个 Transformer 块，最后应用一个可学习的线性投影（“输出嵌入”或“LM 头”）来生成预测的下一词元 logits。示意结构见图 1。

token IDs to dense vectors, passes the embedded tokens through `num_layers` Transformer blocks, and th
applies a learned linear projection (the "output embedding" or "LM head") to produce the predicted
next-token logits. See Figure 1 for a schematic representation.

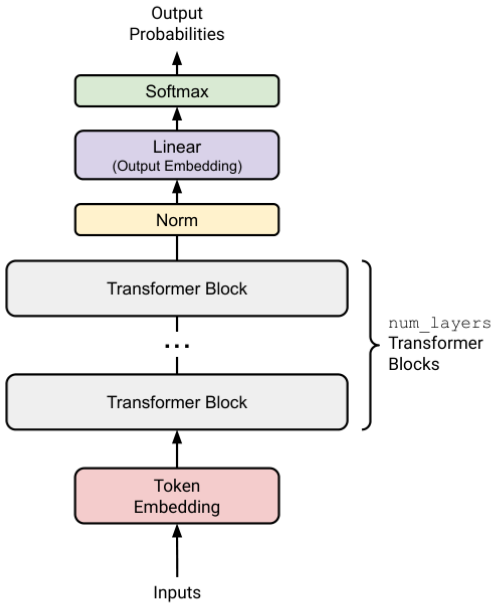

Figure 1: An overview of our Transformer language model.

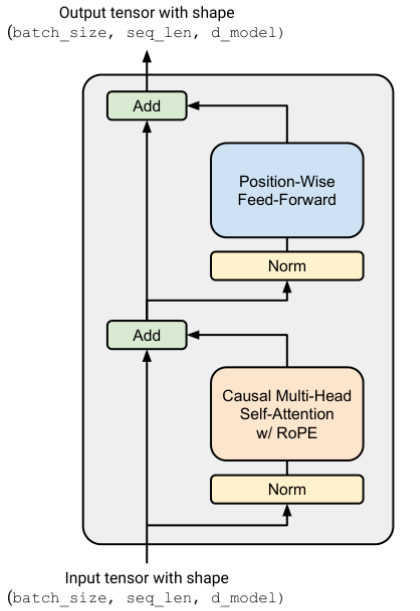

Figure 2: A pre-norm Transformer block.

Figure 1 and Figure 2 from source PDF page 13

Figure 1: An overview of our Transformer language model. Figure 2: A pre-norm Transformer block.

图 1：Transformer 语言模型概览。图 2：预归一化 Transformer 块。

Token Embeddings

词元嵌入

In the very first step, the Transformer embeds the batched sequence of token IDs into a sequence of vectors containing information on the token identity (red blocks in Figure 1).

在第一步中，Transformer 将成批的词元 ID 序列嵌入为一组向量；这些向量包含词元身份信息（见图 1 中的红色方块）。

More specifically, given a sequence of token IDs, the Transformer language model uses a token embedding layer to produce a sequence of vectors. Each embedding layer takes in a tensor of integers of shape `(batch_size, sequence_length)` and produces a sequence of vectors of shape `(batch_size, sequence_length, d_model)`.

> 更具体地说，给定一个词元 ID 序列，Transformer 语言模型使用词元嵌入层生成一个向量序列。每个嵌入层接收形状为 `(batch_size, sequence_length)` 的整数张量，并生成形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

Pre-norm Transformer Block

预归一化 Transformer 块

After embedding, the activations are processed by several identically structured neural network layers. A standard decoder-only Transformer language model consists of `num_layers` identical layers, commonly called Transformer "blocks." Each Transformer block takes in an input of shape `(batch_size, sequence_length, d_model)` and returns an output of the same shape. Each block aggregates information across the sequence via self-attention and non-linearly transforms it via the feed-forward layers.

> 完成嵌入后，激活值会经过若干结构相同的神经网络层处理。标准的仅解码器 Transformer 语言模型由 `num_layers` 个相同的层组成，这些层通常称为 Transformer“块”。每个 Transformer 块接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回相同形状的输出。每个块通过自注意力在序列范围内聚合信息，再通过前馈层对信息进行非线性变换。

After `num_layers` Transformer blocks, we will take the final activations and turn them into a distribution over the vocabulary.

> 经过 `num_layers` 个 Transformer 块后，我们会取出最终激活值，并将其转换为词表上的概率分布。

We will implement the "pre-norm" Transformer block (detailed in Section 3.4), which additionally requires the use of layer normalization (detailed below) after the final Transformer block to ensure its outputs are properly scaled.

> 我们将实现第 3.4 节详述的“预归一化”Transformer 块。该结构还要求在最后一个 Transformer 块之后使用下文介绍的层归一化，以确保输出的尺度合适。

After this normalization, we will use a standard learned linear transformation to convert the output of the Transformer blocks into predicted next-token logits (see, e.g., A. Radford et al. [7], equation 2).

> 完成归一化后，我们会使用一个标准的可学习线性变换，将 Transformer 块的输出转换为下一词元的预测 logits（例如可参见 A. Radford 等人 [7] 的公式 2）。

3.2 Remark: Batching, Einsum and Efficient Computation

说明：批处理、爱因斯坦求和与高效计算

Throughout the Transformer, we will be performing the same computation on many batch-like inputs. Here are a few examples:

在整个 Transformer 中，我们会对许多类似批次的输入执行相同计算。下面给出几个例子：

- Elements of a batch: we apply the same Transformer forward operation to each batch element.
- Sequence length: "position-wise" operations such as RMSNorm and feed-forward networks operate identically on each position of a sequence.
- Attention heads: the attention operation is batched across attention heads in a "multi-headed" attention operation.

- 批次中的元素：我们对批次中的每个元素应用相同的 Transformer `forward` 运算。
- 序列长度：RMSNorm 和前馈网络等“逐位置”运算，会以完全相同的方式作用于序列中的每个位置。
- 注意力头：在“多头”注意力运算中，注意力计算会跨不同注意力头进行批处理。

It is useful to have an ergonomic way of performing such operations that fully utilizes the GPU and is easy to read and understand. Many PyTorch operations can take extra "batch-like" dimensions at the start of a tensor and efficiently repeat or broadcast the operation across these dimensions.

我们需要一种便于使用的方式来执行这些运算，使其既能充分利用 GPU，又容易阅读和理解。许多 PyTorch 运算可以在张量开头接收额外的“类批次”维度，并高效地在这些维度上重复或广播运算。

For instance, suppose we are performing a position-wise batched operation. We have a data tensor D of shape (batch_size, sequence_length, d_model), and we would like to do a batched vector-matrix multiply against a matrix A of shape (d_model, d_model). In this case, D @ A performs a batched matrix multiplication—an efficient primitive in PyTorch—where the (batch_size, sequence_length) dimensions are treated as batch dimensions.

例如，假设我们正在执行一个逐位置的批处理运算。数据张量 D 的形状为 (batch_size, sequence_length, d_model)，我们希望用它与形状为 (d_model, d_model) 的矩阵 A 做批量向量—矩阵乘法。在这种情况下，D @ A 会执行批量矩阵乘法；这是 PyTorch 中的高效基础运算，其中 (batch_size, sequence_length) 两个维度作为批次维度处理。

Because of this, it is helpful to assume that your functions may be given additional batch-like dimensions and to keep those dimensions at the start of the PyTorch shape. Organizing tensors so they can be batched in this manner may require many steps of `view`, `reshape`, and `transpose`. This can be a bit of a pain, and it often makes the code and tensor shapes hard to read.

> 因此，最好假设函数可能会接收到额外的类批次维度，并将这些维度保留在 PyTorch 形状的开头。为了以这种方式组织并批处理张量，往往需要经过多步 `view`、`reshape` 和 `transpose`。这会带来一些麻烦，也常常使代码的作用和张量形状变得难以阅读。

A more ergonomic option is to use einsum notation within `torch.einsum`, or framework-agnostic libraries such as `einops` or `einx`. The two key operations are `einsum`, which can perform tensor contractions over arbitrary input dimensions, and `rearrange`, which can reorder, concatenate, and split arbitrary dimensions. Almost all machine-learning operations are some combination of dimension juggling and tensor contraction with the occasional, usually pointwise, nonlinear function. As a result, much of your code can be more readable and flexible when written with einsum notation.

> 一种更顺手的选择是在 `torch.einsum` 中使用爱因斯坦求和记法，或者使用 `einops`、`einx` 这类与框架无关的库。其中两个关键运算是 `einsum` 和 `rearrange`：前者可以在输入张量的任意维度上执行张量缩并，后者可以重排、合并与拆分任意维度。机器学习中的几乎所有运算，都是维度调整与张量缩并的某种组合，偶尔再加上通常按元素执行的非线性函数。因此，使用爱因斯坦求和记法可以让大量代码更易读、更灵活。

We strongly recommend learning and using einsum notation for the class. Students who have not encountered einsum notation before should use `einops` (documentation linked in the source), and students already comfortable with `einops` should learn the more general `einx`. Both packages are installed in the supplied environment.

> 我们强烈建议为本课程学习并使用爱因斯坦求和记法。以前没有接触过这种记法的同学应使用 `einops`（原文提供了文档链接）；已经熟悉 `einops` 的同学可以学习更通用的 `einx`。我们提供的环境中已经安装了这两个包。

The following examples supplement the `einops` documentation, which you should read first.

> 下面给出的例子是对 `einops` 文档的补充；你应先阅读该文档。

Example (einstein_example1): Batched matrix multiplication with `einops.einsum`

示例（einstein_example1）：使用 `einops.einsum` 进行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
```

```
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#                         D               A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

Example (einstein_example2): Broadcasted operations with `einops.rearrange`

示例（einstein_example2）：使用 `einops.rearrange` 进行广播运算

We have a batch of images, and for each image we want to generate 10 dimmed versions based on a scaling factor:

我们有一批图像，希望基于某个缩放因子，为每张图像生成 10 个变暗版本：

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,    "dim_value               -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

Example (einstein_example3): Pixel mixing with `einops.rearrange`

示例（einstein_example3）：使用 `einops.rearrange` 混合像素

Suppose we have a batch of images represented as a tensor of shape `(batch, height, width, channel)`, and we want to perform a linear transformation across all pixels of each image, independently for every channel. The linear transformation is represented by a matrix B of shape `(height * width, height * width)`.

假设有一批图像，表示为形状为 `(batch, height, width, channel)` 的张量。我们希望对每张图像的全部像素执行线性变换，并让该变换在每个通道上相互独立。线性变换由形状为 `(height * width, height * width)` 的矩阵 B 表示。

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)

## Instead, using einops:
height = width = 32
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
```

```
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

## Or, if you are feeling adventurous: all in one go with einx.dot
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

> The first implementation could be improved with comments that state the input and output shapes, but this is clunky and susceptible to bugs. With einsum notation, the documentation is the implementation.

> 第一种实现可以通过在前后添加注释来说明输入与输出形状，但这样做很笨重，也容易出错。使用爱因斯坦求和记法时，文档本身就是实现。

Einsum notation can handle arbitrary batching dimensions and has the key benefit of being self-documenting. It is much clearer what the relevant input and output tensor shapes are in code that uses einsum notation. For the remaining tensors, you may also consider tensor type hints, for example through the `jaxtyping` library (which is not specific to JAX).

> 爱因斯坦求和记法既能处理任意批处理维度，也有自文档化这一关键优势。使用这种记法的代码能更清楚地展示输入与输出张量的相关形状。对于其余张量，也可以考虑使用张量类型提示，例如采用并非 JAX 专属的 `jaxtyping` 库。

We will discuss the performance implications of einsum notation in Assignment 2. For now, know that it is almost always better than the alternative.

> 我们会在作业 2 中进一步讨论爱因斯坦求和记法对性能的影响。现阶段只需知道，它几乎总是优于其他写法。

3.2.1 Mathematical Notation and Memory Ordering

数学记号与内存顺序

Many machine-learning papers use row vectors in their notation, producing representations that align well with the row-major memory ordering used by default in NumPy and PyTorch. With row vectors, a linear transformation looks like

> 许多机器学习论文在记号中使用行向量，这种表示与 NumPy 和 PyTorch 默认采用的行主序内存排列十分契合。使用行向量时，线性变换写作

$$y = x W^T.$$

for row-major $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and row vector $x \in \mathbb{R}^{1 \times d_{in}}$. This lets us batch inputs by increasing the outermost dimension of x, so vector input x can be replaced with matrix input $X \in \mathbb{R}^{batch \times d_{in}}$.

其中，$W \in \mathbb{R}^{d_{out} \times d_{in}}$ 按行主序存储，$x \in \mathbb{R}^{1 \times d_{in}}$ 为行向量。这样可以通过扩大 x 的最外层维度来批处理输入，即用矩阵输入 $X \in \mathbb{R}^{batch \times d_{in}}$ 替代向量输入 x。

In linear algebra it is generally more common to use column vectors, where linear transformations look like

在线性代数中，使用列向量通常更常见，此时线性变换写作

$$y = W x .$$

given a row-major $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and column vector $x \in \mathbb{R}^{d_{in}}$. To batch the input in this setting, the batch dimension would have to come last, so x would be replaced with a matrix $X \in \mathbb{R}^{d_{in} \times batch}$.

其中，$W \in \mathbb{R}^{d_{out} \times d_{in}}$ 按行主序存储，$x \in \mathbb{R}^{d_{in}}$ 为列向量。在这种设定下批处理输入时，批次维度必须放在最后，因此需要用矩阵 $X \in \mathbb{R}^{d_{in} \times batch}$ 替代 x。

We will mostly use column vectors for mathematical notation in this assignment, since mathematics generally follows this convention. Keep in mind that if you use ordinary matrix-multiplication notation, matrices must be applied with a transpose as in the row-vector convention of Equation 1 because PyTorch uses row-major memory ordering. If you use einsum for linear-algebra operations, this is not an issue as long as you label the axes correctly. Other languages and linear-algebra packages such as Matlab, Julia, and Fortran use column-major memory ordering, where batching dimensions come last; Python and related packages instead follow the C convention of row-major ordering.

本作业中的数学记号主要采用列向量，因为数学中通常遵循这一惯例。需要注意的是，由于 PyTorch 使用行主序内存排列，如果你想使用普通的矩阵乘法记号，就必须像公式 1 的行向量约定那样对矩阵取转置。若使用爱因斯坦求和执行线性代数运算，只要正确标记各个轴，这就不再是问题。顺带一提，Matlab、Julia 和 Fortran 等其他语言或线性代数包采用列主序内存排列，因此批处理维度位于最后；Python 及相关软件包则采用 C 标准的行主序排列。

3.3 Basic Building Blocks: Linear and Embedding Modules

基础组件：线性模块与嵌入模块

3.3.1 Parameter Initialization

参数初始化

Training neural networks effectively often requires careful parameter initialization: poor initializations can cause undesirable behavior such as vanishing or exploding gradients. Pre-norm Transformers are unusually robust to initialization, but initialization can still significantly affect training speed and convergence. Since this assignment is already long, we defer the details to Assignment 3 and provide approximate initializations that should work well in most cases. For now, use:

> 要有效训练神经网络，往往需要谨慎初始化模型参数；不良初始化可能导致梯度消失或梯度爆炸等问题。预归一化 Transformer 对初始化方式格外稳健，但初始化仍会显著影响训练速度与收敛。由于本作业已经很长，相关细节留到作业 3；这里先给出在大多数情况下都应有效的近似初始化方案：

- Linear weights: $N\left(\mu=0,\sigma^2=\frac{2}{d_{in}+d_{out}}\right)$ truncated to $[-3\sigma,3\sigma)$.
- Embedding: $N\left(\mu=0,\sigma^2=1\right)$ truncated to $[-3,3)$.
- RMSNorm: 1.
- 线性层权重：$N\left(\mu=0,\sigma^2=\frac{2}{d_{in}+d_{out}}\right)$，截断到 $[-3\sigma,3\sigma)$。
- 嵌入：$N\left(\mu=0,\sigma^2=1\right)$，截断到 $[-3,3)$。
- RMSNorm：1。

Use `torch.nn.init.trunc_normal_` to initialize the truncated normal weights.

> 应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 Linear Module

线性模块

Linear layers are a fundamental building block of Transformers and neural networks in general. First, you will implement your own `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation:

> 线性层是 Transformer 乃至一般神经网络的基础构件。首先，你将实现一个继承自 `torch.nn.Module` 的自定义 `Linear` 类，用它执行线性变换：

$$y=W x.$$

Note that, following most modern LLMs, we do not include a bias term.

注意，遵循多数现代大语言模型的做法，这里不包含偏置项。

Problem (linear): Implementing the linear module (1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except that it has no bias argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)`

Construct a linear-transformation module. This function should accept the following parameters:

- `in_features: int` — final dimension of the input
- `out_features: int` — final dimension of the output
- `device: torch.device | None = None` — device on which to store the parameters
- `dtype: torch.dtype | None = None` — data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor`

Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`;
- call the superclass constructor;
- construct and store your parameter as W (not W^T), placing it in an `nn.Parameter`;
- not use `nn.Linear` or `nn.functional.linear`.

For initialization, use the settings above together with `torch.nn.init.trunc_normal_`.

To test your `Linear` module, implement the test adapter at `adapters.run_linear`. The adapter should load the supplied weights into the `Linear` module; you may use `Module.load_state_dict` for this purpose. Then run `uv run pytest -k test_linear`.

`def __init__(self, in_features, out_features, device=None, dtype=None)`
`def forward(self, x: torch.Tensor) -> torch.Tensor`

问题（linear）：实现线性模块（1 分）

交付要求：实现一个继承自 `torch.nn.Module` 并执行线性变换的 `Linear` 类。除去不包含偏置参数或偏置选项之外，实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口。建议采用以下接口：

构造线性变换模块。该函数应接受以下参数：

- `in_features: int` — 输入的最后一个维度
- `out_features: int` — 输出的最后一个维度
- `device: torch.device | None = None` — 存放参数的设备
- `dtype: torch.dtype | None = None` — 参数的数据类型

对输入应用线性变换。

请确保：

- 继承 `nn.Module`；
- 调用父类构造函数；
- 将参数按 W（而不是 W^T）的形式构造并存储到 `nn.Parameter` 中；
- 不使用 `nn.Linear` 或 `nn.functional.linear`。

初始化时，请采用上文给出的设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试 `Linear` 模块，请实现 `adapters.run_linear` 处的测试适配器。适配器应将给定权重加载到 `Linear` 模块中；可使用 `Module.load_state_dict`。然后运行 `uv run pytest -k test_linear`。

3.3.3 Embedding Module

嵌入模块

As discussed above, the first layer of the Transformer is an embedding layer that maps integer token IDs into a vector space of dimension `d_model`. We will implement a custom `Embedding` class that inherits from `torch.nn.Module` (so you should not use `nn.Embedding`). The forward method should select the embedding vector for each token ID by indexing into an embedding matrix of shape `(vocab_size, d_model)` using a `torch.LongTensor` of token IDs with shape `(batch_size, sequence_length)`.

如上所述，Transformer 的第一层是一个嵌入层，将整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个自定义的 `Embedding` 类，该类继承自 `torch.nn.Module`（因此你不应该使用 `nn.Embedding`）。forward 方法应该通过使用形状为 `(batch_size, sequence_length)` 的 `torch.LongTensor` token ID 索引形状为 `(vocab_size, d_model)` 的嵌入矩阵，为每个 token ID 选择嵌入向量。

Problem (embedding): Implement the embedding module (1 point)

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

- `__init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` — Construct an embedding module.
- `forward(self, token_ids: torch.Tensor) -> torch.Tensor` — Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as an `nn.Parameter`

- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `adapters.run_embedding`. Then, run `uv run pytest -k test_embedding`.

问题（embedding）：实现嵌入模块（1 分）

交付物：实现继承自 `torch.nn.Module` 并执行嵌入查找的 `Embedding` 类。你的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们建议以下接口：

- 构造嵌入模块。
  - `num_embeddings: int` — Size of the vocabulary / 词汇表大小
  - `embedding_dim: int` — Dimension of the embedding vectors, i.e., `d_model` / 嵌入向量的维度，即 `d_model`
  - `device: torch.device | None = None` — Device to store the parameters on / 存储参数的设备
  - `dtype: torch.dtype | None = None` — Data type of the parameters / 参数的数据类型
- 为给定的 token ID 查找嵌入向量。

确保：

- 继承 `nn.Module`
- 调用超类构造函数
- 将你的嵌入矩阵初始化为 `nn.Parameter`
- 存储嵌入矩阵，其中 `d_model` 为最后一维
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`

再次，使用上述设置进行初始化，并使用 `torch.nn.init.trunc_normal_` 来初始化权重。

要测试你的实现，实现 `adapters.run_embedding` 处的测试适配器。然后运行 `uv run pytest -k test_embedding`。

3.4 Pre-Norm Transformer Block

Pre-Norm Transformer 块

Each Transformer block has two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network ([A. Vaswani et al., 2017], section 3.1).

> 每个 Transformer 块有两个子层：多头自注意力机制和逐位置前馈网络（[A. Vaswani et al., 2017]，第 3.1 节）。

In the original Transformer paper, the model uses a residual connection around each of the two sub-layers, followed by layer normalization. This architecture is commonly known as the "post-norm" Transformer, since layer normalization is applied to the sub-layer output. However, a variety of work has found that moving layer normalization from the output of each sub-layer to the input of each sub-layer (with an additional layer normalization after the final Transformer block) improves Transformer training stability [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020] – see Figure 2 for a visual representation of this "pre-norm" Transformer block. The output of each Transformer block sub-layer is then added to the sub-layer input via the residual connection (A. Vaswani et al. [8], section 5.4). An intuition for pre-norm is that there is a clean "residual stream" without any normalization going from the input embeddings to the final output of the Transformer, which is purported to improve gradient flow. This pre-norm Transformer is now the standard used in language models today (e.g., GPT-3, LLaMA, PaLM, etc.), so we will implement this variant. We will walk through each of the components of a pre-norm Transformer block, implementing them in sequence.

> 在原始 Transformer 论文中，模型在每个子层周围使用残差连接，然后进行层归一化。这种架构通常被称为”post-norm”Transformer，因为层归一化应用于子层输出。然而，许多研究发现，将层归一化从每个子层的输出移到每个子层的输入（并在最后一个 Transformer 块之后增加一个额外的层归一化）可以提高 Transformer 训练的稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]——参见图 2 了解这种”pre-norm”Transformer 块的可视化表示。然后，每个 Transformer 块子层的输出通过残差连接添加到子层输入（A. Vaswani et al. [8]，第 5.4 节）。pre-norm 的直觉是，从输入嵌入到 Transformer 的最终输出之间有一条干净的”残差流”，没有任何归一化，这被认为能改善梯度流动。这种 pre-norm Transformer 现在是当今语言模型的标准（例如 GPT-3、LLaMA、PaLM 等），因此我们将实现这个变体。我们将逐一讲解 pre-norm Transformer 块的每个组件，并依次实现它们。

3.4.1 Root Mean Square Layer Normalization

均方根层归一化（RMSNorm）

The original Transformer implementation of A. Vaswani et al. [8] uses layer normalization [J. L. Ba et al., 2016] to normalize activations. Following H. Touvron et al. [12], we will use root mean square layer normalization (RMSNorm; B. Zhang et al. [13], equation 4) for layer normalization. Given a vector $x \in \mathbb{R}^{d_{model}}$ of activations, RMSNorm will rescale each activation as follows:

A. Vaswani 等人 [8] 的原始 Transformer 实现使用层归一化 [J. L. Ba et al., 2016] 来归一化激活值。遵循 H. Touvron 等人 [12]，我们将使用均方根层归一化（RMSNorm；B. Zhang et al. [13]，方程 4）进行层归一化。给定激活值向量 $x \in \mathbb{R}^{d_{model}}$，RMSNorm 将按如下方式重新缩放每个激活值：

$$\text{RMSNorm}(x)_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i,$$

where $\text{RMS}(x) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i^2 + \epsilon}$. Here, γ_i is a learnable "gain" parameter (there are d_{model} such parameters total), and ϵ is a hyperparameter that is often fixed at 1e-5.

其中 $\text{RMS}(x) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i^2 + \epsilon}$。这里，$\gamma_i$ 是可学习的”增益”参数（总共有 d_{model} 个这样的参数），ϵ 是一个通常固定为 1e-5 的超参数。

You should upcast your input to `torch.float32` to prevent overflow when you square the input. Overall, your forward method should look like:

你应该将输入向上转换为 `torch.float32`，以防止在平方输入时溢出。总体而言，你的 forward 方法应该如下所示：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement RMSNorm as a `torch.nn.Module`. We recommend the following interface:

- `__init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)` — Construct the RMSNorm module.
- `forward(self, x: torch.Tensor) -> torch.Tensor` — Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `adapters.run_rmsnorm`. Then, run `uv run pytest -k test_rmsnorm`.

问题（rmsnorm）：均方根层归一化（1 分）

交付物：将 RMSNorm 实现为 `torch.nn.Module`。我们建议以下接口：

- 构造 RMSNorm 模块。
  - `d_model: int` — Hidden dimension of the model / 模型的隐藏维度
  - `eps: float = 1e-5` — Epsilon value for numerical stability / 数值稳定性的 epsilon 值
  - `device: torch.device | None = None` — Device to store the parameters on / 存储参数的设备
  - `dtype: torch.dtype | None = None` — Data type of the parameters / 参数的数据类型
- 处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量，并返回相同形状的张量。

注意：记住在执行归一化之前将输入向上转换为 `torch.float32`（然后向下转换回原始 dtype），如上所述。

要测试你的实现，实现 `adapters.run_rmsnorm` 处的测试适配器。然后运行 `uv run pytest -k test_rmsnorm`。

3.4.2 Position-Wise Feed-Forward Network

逐位置前馈网络

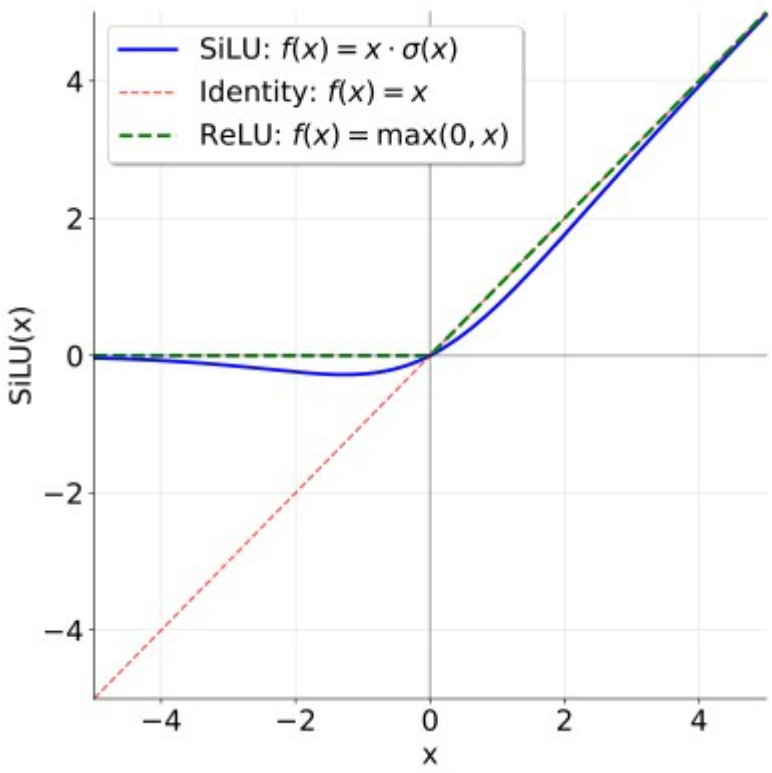

Figure 3: Comparing SiLU (Swish) and ReLU activation functions.

Figure 3: Comparing the SiLU (aka Swish) and ReLU activation functions.

图 3：SiLU（也称 Swish）与 ReLU 激活函数的比较。

In the original Transformer paper (section 3.3 of A. Vaswani et al. [8]), the Transformer feed-forward network consists of two linear transformations with a ReLU activation ($\mathrm{ReLU}(x)=max(0,x)$) between them. In that original architecture, the dimensionality of the inner feed-forward layer is typically 4x the input dimensionality.

在原始 Transformer 论文（A. Vaswani et al. [8] 第 3.3 节）中，Transformer 前馈网络由两个线性变换组成，中间有一个 ReLU 激活函数（$\mathrm{ReLU}(x)=max(0,x)$）。在原始架构中，内部前馈层的维度通常是输入维度的 4 倍。

However, modern language models tend to incorporate two main changes compared to this original design: they use another activation function and employ a gating mechanism. Specifically, we will implement the “SwiGLU” activation function adopted in LLMs like Llama 3 [A. Grattafiori et al., 2024] and Qwen 2.5 [A. Yang et al., 2024], which combines the SiLU (often called Swish) activation with a gating mechanism called a Gated Linear Unit (GLU). We will also omit the bias terms sometimes used in linear layers, following most modern LLMs since PaLM [A. Chowdhery et al., 2022] and LLaMA [H. Touvron et al., 2023].

然而，现代语言模型相比原始设计通常包含两个主要变化：它们使用另一种激活函数并采用门控机制。具体来说，我们将实现在 Llama 3 [A. Grattafiori et al., 2024] 和 Qwen 2.5 [A. Yang et al., 2024] 等 LLM 中采用的”SwiGLU”激活函数，它将 SiLU（通常称为 Swish）激活函数与称为门控线性单元（GLU）的门控机制相结合。我们也将省略线性层中有时使用的偏置项，这是遵循自 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来大多数现代 LLM 的做法。

The SiLU or Swish activation function [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] is defined as follows:

SiLU 或 Swish 激活函数 [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义如下：

$$\mathrm{SiLU}(x)=x\cdot\sigma(x)=\frac{x}{1+e^{-x}}$$

As can be seen in Figure 3, the SiLU activation function is similar to the ReLU activation function, but is smooth at zero.

如图 3 所示，SiLU 激活函数类似于 ReLU 激活函数，但在零点处是平滑的。

Gated Linear Units (GLUs) were originally defined by Y. N. Dauphin et al. [19] as the element-wise product of a linear transformation passed through a sigmoid function and another linear transformation:

门控线性单元（GLU）最初由 Y. N. Dauphin 等人 [19] 定义为通过 sigmoid 函数的线性变换与另一线性变换的逐元素乘积：

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(xW_1) \odot xW_2,$$

where ⊙ represents element-wise multiplication. Gated Linear Units are suggested to "reduce the vanishing gradient problem for deep architectures by providing a linear path for the gradients while retaining non-linear capabilities."

其中 ⊙ 表示逐元素乘法。门控线性单元被认为"通过为梯度提供线性路径，同时保留非线性能力，来减少深层架构中的梯度消失问题。"

Putting the SiLU/Swish and GLU together, we get the SwiGLU, which we will use for our feed-forward networks:

将 SiLU/Swish 和 GLU 结合，我们得到了 SwiGLU，它将用于我们的前馈网络：

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = (\mathrm{SiLU}(xW_1) \odot xW_3)W_2,$$

where $W_1, W_3 \in \mathbb{R}^{d_{model} \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$ and canonically, $d_{ff} = \frac{8}{3} d_{model}$. For concrete implementations, it is fine to round this to a nearby multiple of 64 for hardware efficiency.

其中 $W_1, W_3 \in \mathbb{R}^{d_{model} \times d_{ff}}$，$W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$，且规范地，$d_{ff} = \frac{8}{3} d_{model}$。对于具体实现，可以将其舍入到最近的 64 的倍数以提高硬件效率。

N. Shazeer [20] first proposed combining the SiLU/Swish activation with GLUs and conducted experiments showing that SwiGLU outperforms baselines like ReLU and SiLU (without gating) on language modeling tasks. Later in the assignment, you will compare SwiGLU and SiLU. Though we've mentioned some heuristic arguments for these components (and the papers provide more supporting evidence), it's good to keep an empirical perspective: a now famous quote from Shazeer's paper is

N. Shazeer [20] 首先提出将 SiLU/Swish 激活与 GLU 结合，并进行了实验，表明 SwiGLU 在语言建模任务上优于 ReLU 和 SiLU（无门控）等基线。在作业的后面部分，你将比较 SwiGLU 和 SiLU。虽然我们已经为这些组件提到了一些启发式论证（并且论文提供了更多的支持证据），但保持经验视角是好的：

Shazeer 论文中现在著名的一句话是

> "We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence."

"我们不解释为什么这些架构似乎有效；我们将它们的成功，如同其他一切一样，归因于神圣的仁慈。"

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{model}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `adapters.run_swiglu`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

问题（positionwise_feedforward）：实现逐位置前馈网络（2 分）

交付物：实现 SwiGLU 前馈网络，由 SiLU 激活函数和 GLU 组成。

注意：在这种特定情况下，你可以自由地在实现中使用 `torch.sigmoid` 以获得数值稳定性。

在你的实现中，你应该将 d_{ff} 设置为大约 $\frac{8}{3} \times d_{model}$，同时确保内部前馈层的维度是 64 的倍数，以充分利用你的硬件。要根据我们提供的测试来测试你的实现，你需要实现 `adapters.run_swiglu` 处的测试适配器。然后运行 `uv run pytest -k test_swiglu` 来测试你的实现。

3.4.3 Relative Positional Embeddings

相对位置嵌入（RoPE）

To inject positional information into the model, we will implement Rotary Position Embeddings [J. Su et al., 2021], often called RoPE. For a given query token $q^{(i)} = \left(q_1^{(i)}, \ldots, q_d^{(i)}\right)$ at token position i, we will apply a pairwise rotation matrix R_i, giving us $\tilde{q}^{(i)} = R_i q^{(i)}$. Here, R_i will rotate pairs of embedding elements as 2d vectors by the angle $\theta_{i,j} = i \cdot \theta^{-(2j-2)/d}$ for $j \in \{1, \ldots, d/2\}$ and some constant θ. Thus, we can consider R_i to be a block-diagonal matrix of size $d \times d$, with blocks $R_{i,j}$ for $j \in \{1, \ldots, \frac{d}{2}\}$, with

为了将位置信息注入到模型中，我们将实现旋转位置嵌入 [J. Su et al., 2021]，通常称为 RoPE。对于 token 位置 i 处的给定查询 token $q^{(i)} = \left(q_1^{(i)}, \ldots, q_d^{(i)}\right)$，我们将应用逐对旋转矩阵 R_i，得到 $\tilde{q}^{(i)} = R_i q^{(i)}$。这里，$R_i$ 将嵌入元素的逐对元素作为 2d 向量，按角度 $\theta_{i,j} = i \cdot \theta^{-(2j-2)/d}$ 旋转，其中 $j \in \{1, \ldots, d/2\}$，θ 为某个常数。因此，我们可以将 R_i 视为大小为 $d \times d$ 的块对角矩阵，对于 $j \in \{1, \ldots, \frac{d}{2}\}$，有块 $R_{i,j}$：

$$R_{i,j} = \begin{pmatrix} \cos\left(\theta_{i,j}\right) & -\sin\left(\theta_{i,j}\right) \\ \sin\left(\theta_{i,j}\right) & \cos\left(\theta_{i,j}\right) \end{pmatrix}$$

Thus we get the full rotation matrix

因此我们得到完整的旋转矩阵

$$R_i = \begin{pmatrix} R_{i,1} & 0 & \cdots & 0 \\ 0 & R_{i,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R_{i,d/2} \end{pmatrix}$$

where 0s represent 2×2 zero matrices. While one could construct the full $d \times d$ matrix, a good solution should use the properties of this matrix to implement the transformation more efficiently. Since we only care about the relative rotation of tokens within a given sequence, we can reuse the values we compute for $\cos\left(\theta_{i,j}\right)$ and $\sin\left(\theta_{i,j}\right)$ across layers, and different batches. If you would like to optimize it, you may use a single RoPE module referenced by all layers, and it can have a 2d pre-computed buffer of sin and cos values created during init with self.register_buffer(persistent=False), instead of an nn.Parameter (because we do not want to learn these fixed cosine and sine values). The exact same rotation process we did for our $q^{(i)}$ is then done for $k^{(i)}$, rotating by the corresponding $\theta_{i,j}$. Notice that this layer has no learnable parameters.

其中 0 表示 2×2 零矩阵。虽然可以构造完整的 $d \times d$ 矩阵，但一个好的解决方案应该利用该矩阵的性质更高效地实现变换。由于我们只关心给定序列中 token 的相对旋转，我们可以在不同层和不同批次之间重用为 $\cos\left(\theta_{i,j}\right)$ 和 $\sin\left(\theta_{i,j}\right)$ 计算的值。如果你想优化它，你可以使用一个被所有层引用的单个 RoPE 模块，它可以在 init 期间使用 self.register_buffer(persistent=False) 创建一个 2d 预计算的 sin 和 cos 值缓冲区，而不是 nn.Parameter（因为我们不想学习这些固定的余弦和正弦值）。我们对 $q^{(i)}$ 所做的完全相同旋转过程然后对 $k^{(i)}$ 进行，按相应的 $\theta_{i,j}$ 旋转。请注意，该层没有可学习参数。

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor. The following interface is recommended:

- `__init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` — Construct the RoPE module and create buffers if needed.
- `forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` — Process an input tensor of shape `(..., seq_len, d_k)` and return a tensor of the same shape. Note that you should tolerate x with an arbitrary number of batch dimensions. You should assume that the token positions are a tensor of shape `(..., seq_len)` specifying the token positions of x along the sequence dimension.

You should use the token positions to slice your (possibly precomputed) cos and sin tensors along the sequence dimension.

To test your implementation, complete `adapters.run_rope` and make sure it passes `uv run pytest -k test_rope`.

问题（rope）：实现 RoPE（2 分）

交付物：实现一个 `RotaryPositionalEmbedding` 类，将 RoPE 应用于输入张量。

推荐以下接口：

- 构造 RoPE 模块并在需要时创建缓冲区。
  - `theta: float` — θ value for the RoPE / RoPE 的 θ 值
  - `d_k: int` — dimension of query and key vectors / 查询和键向量的维度

- `max_seq_len: int` — Maximum sequence length that will be input / 将输入的最大序列长度
- `device: torch.device | None = None` — Device to store the buffer on / 存储缓冲区的设备

• 处理形状为 `(..., seq_len, d_k)` 的输入张量并返回相同形状的张量。注意，你应该容忍具有任意数量批处理维度的 x。你应该假设 token 位置是形状为 `(..., seq_len)` 的张量，指定 x 沿序列维度的 token 位置。

你应该使用 token 位置沿序列维度切片你的（可能预计算的）cos 和 sin 张量。

要测试你的实现，完成 `adapters.run_rope` 并确保它通过 `uv run pytest -k test_rope`。

3.4.4 Scaled Dot-Product Attention

缩放点积注意力

We will now implement scaled dot-product attention as described in A. Vaswani et al. [8] (section 3.2.1). As a preliminary step, the definition of the Attention operation will make use of softmax, an operation that takes an unnormalized vector of scores and turns it into a normalized distribution:

现在我们将实现 A. Vaswani 等人 [8]（第 3.2.1 节）中描述的缩放点积注意力。作为预备步骤，注意力操作的定义将使用 softmax，这是一个将未归一化的分数向量转换为归一化分布的操作：

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{n} \exp(z_j)}.$$

Note that $\exp(z_i)$ can become inf for large values (then, $\frac{inf}{inf} = \text{NaN}$). We can avoid this by noticing that the softmax operation is invariant to adding any constant c to all inputs. We can leverage this property for numerical stability–typically, we will subtract the largest entry of z from all elements of z, making the new largest entry 0. You will now implement softmax, using this trick for numerical stability.

注意，对于大值，$\exp(z_i)$ 可能变为 inf（然后，$\frac{inf}{inf} = \text{NaN}$）。我们可以通过注意到 softmax 操作对向所有输入添加任意常数 c 是不变的来避免这种情况。我们可以利用此属性获得数值稳定性——通常，我们将从 z 的所有元素中减去 z 的最大条目，使新的最大条目为 0。现在你将使用这个技巧实现 softmax，以保证数值稳定性。

Problem (softmax): Implement softmax (1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a dimension d, and apply softmax to the d-th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its d-th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the d-th dimension from all elements of the d-th dimension to avoid numerical stability issues.

To test your implementation, complete `adapters.run_softmax` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

问题（softmax）：实现 softmax（1 分）

交付物：编写一个函数，在张量上应用 softmax 操作。你的函数应该接受两个参数：一个张量和一个维度 d，并对输入张量的第 d 维应用 softmax。输出张量应与输入张量具有相同的形状，但其第 d 维现在将具有归一化的概率分布。使用从第 d 维的所有元素中减去第 d 维的最大值的技巧来避免数值稳定性问题。

要测试你的实现，完成 `adapters.run_softmax` 并确保它通过 `uv run pytest -k test_softmax_matches_pytorch`。

We can now define the Attention operation mathematically as follows:

现在我们可以数学上定义注意力操作如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, and $V \in \mathbb{R}^{m \times d_v}$. Here, n, m and d_k are all inputs to this operation–note that these are not the learnable parameters.

其中 $Q \in \mathbb{R}^{n \times d_k}$，$K \in \mathbb{R}^{m \times d_k}$，$V \in \mathbb{R}^{m \times d_v}$。这里，$n$、$m$ 和 d_k 都是该操作的输入——注意这些不是可学习参数。

Masking: It is sometimes convenient to mask the output of an attention operation. A mask should have the shape $M \in \{\text{True}, \text{False}\}^{n \times m}$, and each row i of this boolean matrix indicates which keys the query i should attend to. Canonically (and slightly confusingly), a value of True at position (i, j) indicates that the query i does attend to the key j, and a value of False indicates that the query does not attend to the key. In other words, "information flows" at (i, j) pairs with value True. For example, consider a 1×3 mask matrix with entries [[True, True, False]]. The single query vector attends only to the first two keys.

掩码：有时方便地掩码注意力操作的输出。掩码应具有形状 $M \in \{\text{True}, \text{False}\}^{n \times m}$，这个布尔矩阵的每一行 i 表示查询 i 应该关注哪些键。规范地（且有点令人困惑），位置 (i, j) 处的 True 值表示查询 i **确实**关注键 j，False 值表示查询不关注该键。换句话说，“信息流动”在值为 True 的 (i, j) 对处。例如，考虑一个包含条目 [[True, True, False]] 的 1×3 掩码矩阵。单个查询向量只关注前两个键。

Computationally, it will be much more efficient to use masking than to compute attention on subsequences, and we can do this by taking the pre-softmax values ($QK^T/\sqrt{d_k}$) and adding a $-\infty$ to any entry of the mask matrix that is False.

在计算上，使用掩码比在子序列上计算注意力要高效得多，我们可以通过在 softmax 之前的值（$QK^T/\sqrt{d_k}$）中，对掩码矩阵中为 False 的任何条目加上 $-\infty$ 来实现。

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., seq_len, d_v)`. See Section 3.2 for a discussion on batch-like dimensions.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `adapters.run_scaled_dot_product_attention`. `uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention` tests your implementation on fourth-order input tensors.

问题（scaled_dot_product_attention）：实现缩放点积注意力（5 分）

交付物：实现缩放点积注意力函数。你的实现应该处理形状为 `(batch_size, ..., seq_len, d_k)` 的键和查询，以及形状为 `(batch_size, ..., seq_len, d_v)` 的值，其中 `...` 表示任意数量的其他批处理类维度（如果提供）。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。关于批处理类维度的讨论，请参见第 3.2 节。

你的实现还应支持可选的用户提供的布尔掩码，形状为 `(seq_len, seq_len)`。掩码值为 `True` 的位置的注意力概率应总和为 1，掩码值为 `False` 的位置的注意力概率应为零。

要根据我们提供的测试来测试你的实现，你需要实现 `adapters.run_scaled_dot_product_attention` 处的测试适配器。`uv run pytest -k test_scaled_dot_product_attention` 在三阶输入张量上测试你的实现，而 `uv run pytest -k test_4d_scaled_dot_product_attention` 在四阶输入张量上测试你的实现。

3.4.5 Causal Multi-Head Self-Attention

因果多头自注意力

We will implement multi-head self-attention as described in section 3.2.2 of A. Vaswani et al. [8]. Recall that, mathematically, the operation of applying multi-head attention is defined as follows:

我们将实现 A. Vaswani 等人 [8] 第 3.2.2 节中描述的多头自注意力。回顾一下，数学上，应用多头注意力的操作定义如下：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W_O$$

$$\text{for head}_i = \text{Attention}\left(Q W_{Q,i}, K W_{K,i}, V W_{V,i}\right)$$

with $W_{Q,i}, W_{K,i}, W_{V,i}$ being slice number $i \in \{1, \ldots, h\}$ of size d_k or d_v of the embedding dimension for Q, K, and V respectively. With Attention being the scaled dot-product attention operation defined in Section 3.4.4.

其中 $W_{Q,i}, W_{K,i}, W_{V,i}$ 分别是 Q、K 和 V 的嵌入维度中大小为 d_k 或 d_v 的第 $i \in \{1, \ldots, h\}$ 个切片。Attention 是在第 3.4.4 节中定义的缩放点积注意力操作。

From this we can form the multi-head self-attention operation:

由此我们可以形成多头自注意力操作：

$$\text{MultiHeadSelfAttention}\left(X\right) = \text{MultiHead}\left(X, X, X\right)$$

Here, the learnable parameters are $W_Q \in \mathbb{R}^{d_{model} \times d_{model}}$, $W_K \in \mathbb{R}^{d_{model} \times d_{model}}$, $W_V \in \mathbb{R}^{d_{model} \times d_{model}}$ and $W_O \in \mathbb{R}^{d_{model} \times d_{model}}$. Since the Qs, Ks, and Vs are sliced in the multi-head attention operation, we can think of W_Q, W_K and W_V as being separated for each head along the output dimension. When you have this working, you should be computing the key, value, and query projections in a total of three matrix multiplies.

这里，可学习参数是 $W_Q \in \mathbb{R}^{d_{model} \times d_{model}}$，$W_K \in \mathbb{R}^{d_{model} \times d_{model}}$，$W_V \in \mathbb{R}^{d_{model} \times d_{model}}$ 和 $W_O \in \mathbb{R}^{d_{model} \times d_{model}}$。由于 Q、K 和 V 在多头注意力操作中被切片，我们可以将 W_Q、W_K 和 W_V 视为沿输出维度为每个头分开的。当你使其工作时，你应该通过总共三次矩阵乘法来计算键、值和查询投影。

Causal masking

因果掩码

Your implementation should prevent the model from attending to future tokens in the sequence. In other words, if the model is given a token sequence $x_1, \ldots, x_T$, and we want to calculate the next-word predictions for the prefix $x_1, \ldots, x_t$ (where $t < T$), the model should not be able to access (attend to) the token representations at positions $x_{t+1}, \ldots, x_T$ since it will not have access to these tokens when generating text during inference (and these future tokens leak information about the identity of the true next word, trivializing the language modeling pre-training objective). For an input token sequence $x_1, \ldots, x_T$ we can naively prevent access to future tokens by running multi-head self-attention T times (for the T unique prefixes in the sequence). Instead, we'll use causal attention masking, which allows token i to attend to all positions $\leq i$ in the sequence. You can use torch.triu or a broadcasted index comparison to construct this mask, and you should take advantage of the fact that your scaled dot-product attention implementation from Section 3.4.4 already supports attention masking.

你的实现应该防止模型关注序列中未来的 token。换句话说，如果模型被给定一个 token 序列 $x_1, \ldots, x_T$，我们想要计算前缀 $x_1, \ldots, x_t$（其中 $t < T$）的下一个词预测，模型不应该能够访问（关注）位置 $x_{t+1}, \ldots, x_T$ 的 token 表示，因为在推理时生成文本时它将无法访问这些 token（并且这些未来 token 泄露了关于真实下一个词身份的信息，使语言建模预训练目标变得平凡）。对于输入 token 序列 $x_1, \ldots, x_T$，我们可以通过运行多头自注意力 T 次（针对序列中的 T 个唯一前缀）来朴素地阻止对未来 token 的访问。相反，我们将使用因果注意力掩码，它允许 token i 关注序列中所有位置 $\leq i$。你可以使用 torch.triu

或广播索引比较来构造此掩码，并且你应该利用你在第 3.4.4 节中的缩放点积注意力实现已经支持注意力掩码的事实。

Applying RoPE

应用 RoPE

RoPE should be applied to the query and key vectors, but not the value vectors. Also, the head dimension should be handled as a batch dimension, because in multi-head attention, attention is being applied independently for each head. This means that precisely the same RoPE rotation should be applied to the query and key vectors for each head.

RoPE 应应用于查询和键向量，但不应用于值向量。此外，头维度应作为批处理维度处理，因为在多头注意力中，注意力是独立为每个头应用的。这意味着应该对每个头的查询和键向量应用完全相同的 RoPE 旋转。

Problem (multihead_self_attention): Implement causal multi-head self-attention (5 points)

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

- `d_model: int` — Dimensionality of the Transformer block inputs.
- `num_heads: int` — Number of heads to use in multi-head self-attention.

Following A. Vaswani et al. [8], set $d_k = d_v = \frac{d_{model}}{num\_heads}$. To test your implementation against our provided tests, implement the test adapter at `adapters.run_multihead_self_attention`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

问题（multihead_self_attention）：实现因果多头自注意力（5 分）

交付物：将因果多头自注意力实现为 `torch.nn.Module`。你的实现应该（至少）接受以下参数：

- Transformer 块输入的维度。
- 多头自注意力中使用的头数。

遵循 A. Vaswani 等人 [8]，设置 $d_k = d_v = \frac{d_{model}}{num\_heads}$。要根据我们提供的测试来测试你的实现，实现 `adapters.run_multihead_self_attention` 处的测试适配器。然后运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 The Full Transformer LM

完整的 Transformer 语言模型

Let's begin by assembling the Transformer block (it will be helpful to refer back to Figure 2). A Transformer block contains two 'sub-layers', one for the multihead self attention, and another for the SwiGLU feed-forward network. In each sub-layer, we first perform RMSNorm, then the main operation (MHA/FF), finally adding in the residual connection.

让我们从组装 Transformer 块开始（回顾图 2 会很有帮助）。一个 Transformer 块包含两个”子层”，一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，我们首先执行 RMSNorm，然后是主要操作（MHA/FF），最后添加残差连接。

To be concrete, the first half (the first 'sub-layer') of the Transformer block should be implementing the following set of updates to produce an output y from an input x,

具体来说，Transformer 块的前半部分（第一个”子层”）应该实现以下更新集合，从输入 x 产生输出 y：

$$y = x + \text{MultiHeadSelfAttention}\left(\text{RMSNorm}\left(x\right)\right).$$

Problem (transformer_block): Implement the Transformer block (3 points)

Implement the pre-norm Transformer block as described in Section 3.4 and illustrated in Figure 2. Your Transformer block should accept (at least) the following parameters.

- `d_model: int` — Dimensionality of the Transformer block inputs.
- `num_heads: int` — Number of heads to use in multi-head self-attention.
- `d_ff: int` — Dimensionality of the position-wise feed-forward inner layer.

To test your implementation, implement the adapter `adapters.run_transformer_block`. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

问题（transformer_block）：实现 Transformer 块（3 分）

实现第 3.4 节中描述并在图 2 中展示的 pre-norm Transformer 块。你的 Transformer 块应该（至少）接受以下参数。

- Transformer 块输入的维度。
- 多头自注意力中使用的头数。
- 逐位置前馈内部层的维度。

要测试你的实现，实现适配器 `adapters.run_transformer_block`。然后运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

交付物：通过提供测试的 Transformer 块代码。

Now we put the blocks together, following the high-level diagram in Figure 1. Follow our description of the embedding in Section 3.1.0.1, feed this into `num_layers` Transformer blocks, and then pass that into the final layer norm and LM head to obtain an unnormalized distribution over the vocabulary (the logits).

现在我们按照图 1 中的高层图示将这些块组合在一起。遵循我们在第 3.1.0.1 节中对嵌入的描述，将其输入到 `num_layers` 个 Transformer 块中，然后将其传入最终的层归一化和 LM 头，以获得词汇表上的未归一化分布（logits）。

Problem (transformer_lm): Implementing the Transformer LM (3 points)

Time to put it all together! Implement the Transformer language model as described in Section 3.1 and illustrated in Figure 1. At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

- `vocab_size: int` — The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix.
- `context_length: int` — The maximum context length, necessary for determining the dimensionality of the RoPE sin and cos buffer.
- `num_layers: int` — The number of Transformer blocks to use.

To test your implementation against our provided tests, you will first need to implement the test adapter at `adapters.run_transformer_lm`. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

问题（transformer_lm）：实现 Transformer 语言模型（3 分）

是时候将所有内容组合在一起了！实现第 3.1 节中描述并在图 1 中展示的 Transformer 语言模型。至少，你的实现应该接受 Transformer 块的所有上述构造参数，以及以下额外参数：

- 词汇表的大小，确定 token 嵌入矩阵维度所必需的。
- 最大上下文长度，确定 RoPE sin 和 cos 缓冲区的维度所必需的。
- 要使用的 Transformer 块数。

要根据我们提供的测试来测试你的实现，你首先需要实现 `adapters.run_transformer_lm` 处的测试适配器。然后运行 `uv run pytest -k test_transformer_lm` 来测试你的实现。

交付物：通过上述测试的 Transformer LM 模块。

Resource accounting

资源核算

It is useful to be able to understand how the various parts of the Transformer consume compute and memory. We will go through the steps to do some basic "FLOPs accounting." The vast majority of FLOPS in a Transformer are matrix multiplies, so our core approach is simple:

能够理解 Transformer 的各个部分如何消耗计算和内存是很有用的。我们将逐步进行一些基本的"FLOPs 核算"。Transformer 中绝大多数的 FLOPs 是矩阵乘法，因此我们的核心方法很简单：

1. Write down all the matrix multiplies in a Transformer forward pass.

   写下 Transformer 前向传播中的所有矩阵乘法。

2. Convert each matrix multiply into FLOPs required.

   将每个矩阵乘法转换为所需的 FLOPs。

For this second step, the following fact will be useful:

对于第二步，以下事实将很有用：

Rule: Given $A \in \mathbb{R}^{n \times k}$ and $B \in \mathbb{R}^{k \times m}$, the matrix-matrix product AB requires $2nkm$ FLOPs.

规则：给定 $A \in \mathbb{R}^{n \times k}$ 和 $B \in \mathbb{R}^{k \times m}$，矩阵-矩阵乘积 AB 需要 $2nkm$ FLOPs。

To see this, note that $(AB)[i,j] = A[i,:] \cdot B[:,j]$, and that this dot product requires k additions and k multiplications ($2k$ FLOPs). Then, since the matrix-matrix product AB has $n \times m$ entries, the total number of FLOPS is $(2k)(nm) = 2nkm$.

对此的理解是，$(AB)[i,j] = A[i,:] \cdot B[:,j]$，这个点积需要 k 次加法和 k 次乘法（$2k$ FLOPs）。然后，由于矩阵-矩阵乘积 AB 具有 $n \times m$ 个条目，总 FLOPs 数为 $(2k)(nm) = 2nkm$。

Now, before you do the next problem, it can be helpful to go through each component of your Transformer block and Transformer LM, and list out all the matrix multiplies and their associated FLOPs costs.

现在，在做下一个问题之前，浏览你 Transformer 块和 Transformer LM 的每个组件，列出所有矩阵乘法及其相关的 FLOPs 成本，会很有帮助。

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

(a) Consider a GPT-2 XL-sized model using our assignment architecture, which has the following configuration:

- `vocab_size: 50,257`
- `context_length: 1,024`
- `num_layers: 48`

- `d_model: 1,600`
- `num_heads: 25`
- `d_ff: 4,288` (the nearest multiple of 64 to $\frac{8}{3} \times 1,600$)

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Deliverable: A one-to-two sentence response.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Deliverable: A list of matrix multiplies (with descriptions), and the total number of FLOPs required.

(c) Based on your analysis above, which parts of the model require the most FLOPs?

Deliverable: A one-to-two sentence response.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 d_model, 12 heads), GPT-2 medium (24 layers, 1024 d_model, 16 heads), and GPT-2 large (36 layers, 1280 d_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How does the relative contribution of FLOPs of the model components change?

Deliverable: A one-to-two sentence response.

问题（transformer_accounting）：Transformer LM 资源核算（5 分）

考虑一个使用我们作业架构的 GPT-2 XL 大小模型，具有以下配置：

假设我们使用此配置构建了我们的模型。我们的模型将有多少可训练参数？假设每个参数使用单精度浮点表示，仅加载此模型需要多少内存？

交付物：一到两句话回答。

识别完成 GPT-2 XL 形状模型前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设我们的输入序列具有 `context_length` 个 token。

交付物：矩阵乘法列表（带描述），以及所需的 FLOPs 总数。

基于你上面的分析，模型的哪些部分需要最多的 FLOPs？

交付物：一到两句话回答。

使用 GPT-2 small（12 层，768 d_model，12 头）、GPT-2 medium（24 层，1024 d_model，16 头）和 GPT-2 large（36 层，1280 d_model，20 头）重复你的分析。随着模型大小增加，Transformer LM 的哪些部分在总 FLOPs 中占的比例增加或减少？

交付物：对于每个模型，提供模型组件及其相关 FLOPs 的分解（占前向传播所需总 FLOPs 的比例）。此外，提供一到两句话描述模型大小变化如何改变每个组件的比例 FLOPs。

取 GPT-2 XL 并将上下文长度增加到 16,384。一次前向传播的总 FLOPs 如何变化？模型组件的 FLOPs 相对贡献如何变化？

交付物：一到两句话回答。

---

4 Training a Transformer LM

训练 Transformer 语言模型

We now have the steps to preprocess the data (via tokenizer) and the model (Transformer). What remains is to build all of the code to support training. This consists of the following:

我们现在有了预处理数据的步骤（通过分词器）和模型（Transformer）。剩下的就是构建支持训练的所有代码。这包括以下内容：

- **Loss:** we need to define the loss function (cross-entropy).

  损失：我们需要定义损失函数（交叉熵）。

- **Optimizer:** we need to define the optimizer to minimize this loss (AdamW).

  优化器：我们需要定义优化器来最小化这个损失（AdamW）。

- **Training loop:** we need all the supporting infrastructure that loads data, saves checkpoints, and manages training.

  训练循环：我们需要所有支持基础设施，用于加载数据、保存检查点和管理训练。

4.1 Cross-entropy loss

交叉熵损失

Recall that the Transformer language model defines a distribution $p(x_{t+1}|x_{1:t})$ for each sequence x of length $T+1$ and $t=1,\ldots,T$. Given a training set D consisting of sequences of length $T+1$, we define the standard cross-entropy (negative log-likelihood) loss function:

回顾一下，Transformer 语言模型对每个长度为 $T+1$ 的序列 x 和 $t=1,\ldots,T$，定义了一个分布 $p(x_{t+1}|x_{1:t})$。给定由长度为 $T+1$ 的序列组成的训练集 D，我们定义标准的交叉熵（负对数似然）损失函数：

$$L(D;\theta)=\frac{1}{|D|}\sum_{x\in D}\frac{1}{T}\sum_{t=1}^{T}-\log p_\theta(x_{t+1}|x_{1:t}).$$

(Note that a single forward pass in the Transformer yields $p_\theta(x_{t+1}|x_{1:t})$ for all $t=1,\ldots,T$.)

（注意，Transformer 中的单次前向传播可以产生所有 $t=1,\ldots,T$ 的 $p_\theta(x_{t+1}|x_{1:t})$。）

In particular, the Transformer computes logits $\ell\in\mathbb{R}^{vocab\_size}$ for each position t, which results in:

特别地，Transformer 为每个位置 t 计算 logits $\ell\in\mathbb{R}^{vocab\_size}$，这导致：

$$p_\theta(x_{t+1}|x_{1:t})=\text{softmax}(\ell)[x_{t+1}]=\frac{\exp(\ell[x_{t+1}])}{\sum_{j=1}^{vocab\_size}\exp(\ell[j])}.$$

The cross-entropy loss is generally defined with respect to the vector of logits $\ell\in\mathbb{R}^{vocab\_size}$ and target x_{t+1}.

交叉熵损失通常相对于 logits 向量 $\ell\in\mathbb{R}^{vocab\_size}$ 和目标 x_{t+1} 来定义。

Implementing the cross-entropy loss requires some care with numerical issues, just like in the case of softmax.

实现交叉熵损失需要对数值问题给予一些关注，就像 softmax 的情况一样。

Problem (cross_entropy): Implement cross-entropy (1 point)

Deliverable: Write a function to compute the cross-entropy loss, which takes in predicted logits (ℓ) and targets (x_{t+1}) and computes the cross-entropy $\ell_{ce}=-\log\text{softmax}(\ell)[x_{t+1}]$. Your function should handle the following:

- Subtract the largest element for numerical stability.
- Cancel out $\log$ and $\exp$ whenever possible.
- Handle any additional batch dimensions and return the average across the batch. As with Section 3.2, we assume batch-like dimensions always come first, before the vocabulary size dimension.

Implement `adapters.run_cross_entropy`, then run `uv run pytest -k test_cross_entropy` to test your implementation.

问题（cross_entropy）：实现交叉熵（1 分）

交付物：编写一个函数来计算交叉熵损失，该函数接受预测的 logits（ℓ）和目标（x_{t+1}），并计算交叉熵 $\ell_{ce}=-\log\text{softmax}(\ell)[x_{t+1}]$。你的函数应该处理以下内容：

- 减去最大元素以确保数值稳定性。
- 尽可能抵消 $\log$ 和 $\exp$。
- 处理任何额外的批处理维度并返回批处理中的平均值。与第 3.2 节一样，我们假设批处理类维度始终在词汇表大小维度之前出现。

实现 `adapters.run_cross_entropy`，然后运行 `uv run pytest -k test_cross_entropy` 来测试你的实现。

Perplexity

困惑度

Cross-entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length T where we suffer cross-entropy losses $\ell_1, \ldots, \ell_T$:

交叉熵足以用于训练，但当我们评估模型时，我们还希望报告困惑度。对于长度为 T 的序列，其中我们遭受交叉熵损失 $\ell_1, \ldots, \ell_T$：

$$\text{perplexity} = \exp\left(\frac{1}{T}\sum_{t=1}^{T}\ell_t\right).$$

4.2 The SGD Optimizer

SGD 优化器

Now that we have a loss function, we will begin our exploration of optimizers. The simplest gradient-based optimizer is Stochastic Gradient Descent (SGD). We start with randomly initialized parameters θ_0. Then for each step $t = 0, \ldots, T-1$, we perform the following update:

现在我们有了损失函数，我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降（SGD）。我们从随机初始化的参数 θ_0 开始。然后对于每个步骤 $t = 0, \ldots, T-1$，我们执行以下更新：

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla L(B_t; \theta_t),$$

where B_t is a random batch of data sampled from the dataset D, and the learning rate η and batch size $|B_t|$ are hyperparameters.

其中 B_t 是从数据集 D 中采样的随机数据批次，学习率 η 和批次大小 $|B_t|$ 是超参数。

4.2.1 Implementing SGD in PyTorch

在 PyTorch 中实现 SGD

To implement our optimizers, we will subclass the PyTorch `torch.optim.Optimizer` class. An `Optimizer` subclass must implement two methods:

为了实现我们的优化器，我们将子类化 PyTorch 的 `torch.optim.Optimizer` 类。`Optimizer` 子类必须实现两个方法：

- `__init__(self, params, ...)` should initialize your optimizer. Here, `params` will be a collection of parameters to be optimized (or parameter groups, in case the user wants to use different hyperparameters, such as learning rates, for different parts of the model). Make sure to pass `params` to the `__init__` method of the base class, which will store these parameters for use in `step`. You can take additional arguments depending on the optimizer (e.g., the learning rate is a common one), and pass them to the base class constructor as a dictionary, where keys are the names (strings) you choose for these parameters.

> `__init__(self, params, ...)` 应该初始化你的优化器。这里，`params` 将是要优化的参数集合（或参数组，以便用户可以为模型的不同部分使用不同的超参数，如学习率）。确保将 `params` 传递给基类的 `__init__` 方法，该方法将存储这些参数以供 `step` 使用。你可以根据优化器接受额外的参数（例如，学习率是一个常见的参数），并将它们作为字典传递给基类构造函数，其中键是你为这些参数选择的名称（字符串）。

- `step(self)` should make one update of the parameters. During the training loop, this will be called after the backward pass, so you have access to the gradients on the last batch. This method should iterate through each parameter tensor `p` and modify them in place, i.e. setting `p.data`, which holds the tensor associated with that parameter based on the gradient `p.grad` (if it exists), the tensor representing the gradient of the loss with respect to that parameter.

> `step(self)` 应该对参数进行一次更新。在训练循环中，这将在反向传播之后被调用，因此你可以访问最后一批数据的梯度。该方法应该遍历每个参数张量 `p` 并原地修改它们，即设置 `p.data`，它持有基于梯度 `p.grad`（如果存在）与该参数相关联的张量，该张量表示损失相对于该参数的梯度。

The PyTorch optimizer API has a few subtleties, so it's easier to explain it with an example. To make our example richer, we'll implement a slight variation of SGD where the learning rate decays over training, starting with an initial learning rate η and taking successively smaller steps over time:

PyTorch 优化器 API 有一些细微之处，因此通过示例来解释更容易。为了使我们的示例更丰富，我们将实现 SGD 的一个小变体，其中学习率在训练过程中衰减，从初始学习率 η 开始，随时间逐渐减小步长：

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{t+1}} \nabla L\left(B_t; \theta_t\right)$$

Let's see how this version of SGD would be implemented as a PyTorch Optimizer:

> 让我们看看这个版本的 SGD 将如何实现为 PyTorch Optimizer：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
       def __init__(self, params, lr=1e-3):
```

```
                if lr < 0:
                        raise ValueError(f"Invalid learning rate: {lr}")
                defaults = {"lr": lr}
                super().__init__(params, defaults)

        def step(self, closure: Optional[Callable] = None):
                loss = None if closure is None else closure()
                for group in self.param_groups:
                        lr = group["lr"] # Get the learning rate.
                        for p in group["params"]:
                                if p.grad is None:
                                        continue

                                state = self.state[p] # Get state associated with p.
                                t = state.get("t", 0) # Get iteration number from the state, or 0.
                                grad = p.grad.data # Get the gradient of loss with respect to p.
                                p.data -= lr / math.sqrt(t + 1) * grad # Update weight tensor in-
place.
                                state["t"] = t + 1 # Increment iteration number.

                return loss
```

In `__init__`, we pass the parameters, as well as default hyperparameters, to the base class constructor (the parameters might come in groups, each with different hyperparameters). In case the parameters are just a single collection of `torch.nn.Parameter` objects, the base constructor will create a single group and assign it the default hyperparameters. Then, in `step`, we iterate over each parameter group, then over each parameter in that group, and apply Equation 20. Here, we keep the iteration number as a state associated with each parameter: we first read this value, use it in the gradient update, and then update it. The API specifies that the user might pass in a callable `closure` to re-compute the loss before the optimizer step. We won't need this for the optimizers we'll use, but we add it to comply with the API.

在 `__init__` 中，我们将参数以及默认超参数传递给基类构造函数（参数可能以组的形式传入，每个组有不同的超参数）。如果参数只是 `torch.nn.Parameter` 对象的单个集合，基构造函数将创建一个组并为其分配默认超参数。然后，在 `step` 中，我们遍历每个参数组，然后遍历该组中的每个参数，并应用方程 20。这里，我们将迭代次数保持为与每个参数相关联的状态：我们首先读取此值，在梯度更新中使用它，然后更新它。API 规定用户可能传入一个可调用的 `closure`，在优化器步骤之前重新计算损失。我们不会对我们使用的优化器需要这个，但我们添加它是为了符合 API。

To see this working, we can use the following minimal example of a training loop:

为了看到它的工作效果，我们可以使用以下训练循环的最小示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
        opt.zero_grad() # Reset the gradients for all learnable parameters.
        loss = (weights**2).mean() # Compute a scalar loss value.
        print(loss.cpu().item())

        loss.backward() # Run backward pass, which computes gradients.
        opt.step() # Run optimizer step.
```

This is the typical structure of a training loop: in each iteration, we will compute the loss and run a step of the optimizer. When training language models, our learnable parameters will come from the model (in PyTorch, `m.parameters()` gives us this collection). The loss will be computed over a sampled batch of data, but the basic structure of the training loop will be the same.

这是训练循环的典型结构：在每次迭代中，我们将计算损失并运行一次优化器步骤。当训练语言模型时，我们的可学习参数将来自模型（在 PyTorch 中，`m.parameters()` 给我们这个集合）。损失将基于采样的数据批次计算，但训练循环的基本结构将是相同的。

Problem (learning_rate_tuning): Tuning the learning rate (1 point)

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example. Run the SGD example above with three other values for the learning rate: 1e1, 1e2, and 1e3, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?

Deliverable: A one-to-two sentence response with the behaviors you observed.

问题（learning_rate_tuning）：调节学习率（1 分）

正如我们将看到的，影响训练最多的超参数之一是学习率。让我们在我们的玩具示例中实际看看。使用三个其他学习率值：1e1、1e2 和 1e3 运行上述 SGD 示例，仅进行 10 次训练迭代。每个学习率的损失发生了什么变化？它是衰减更快、更慢，还是发散（即在训练过程中增加）？

交付物：一到两句话回答，描述你观察到的行为。

4.3 AdamW

Modern language models are typically trained with more sophisticated optimizers, instead of SGD. Most optimizers used recently are derivatives of the Adam optimizer [D. P. Kingma et al., 2015]. We will use AdamW [I. Loshchilov et al., 2019], which is in wide use in recent work. AdamW proposes a modification to Adam that improves regularization by adding weight decay (at each iteration, we pull the parameters towards 0), in a way that is decoupled from the gradient update. We will implement AdamW as described in algorithm 2 of I. Loshchilov et al. [23].

现代语言模型通常使用更复杂的优化器进行训练，而不是 SGD。最近使用的大多数优化器都是 Adam 优化器 [D. P. Kingma et al., 2015] 的衍生品。我们将使用 AdamW [I. Loshchilov et al., 2019]，它在最近的工作中被广泛使用。AdamW 提出了对 Adam 的修改，通过添加权重衰减（在每次迭代中，我们将参数拉向 0）来改善正则化，其方式与梯度更新解耦。我们将按照 I. Loshchilov et al. [23] 的算法 2 中描述的实现 AdamW。

AdamW is stateful: for each parameter, it keeps track of a running estimate of its first and second moments. Thus, AdamW uses additional memory in exchange for improved stability and convergence. Besides the learning rate η, AdamW has a pair of hyperparameters (β_1, β_2) that control the updates to the moment estimates, and a weight decay rate λ. Typical applications set (β_1, β_2) to $(0.9, 0.999)$, but large language models like LLaMA [H. Touvron et al., 2023] and GPT-3 [T. B. Brown et al., 2020] are often trained with $(0.9, 0.95)$. The algorithm can be written as follows, where ϵ is a small value (e.g., 10^{-8}) used to improve numerical stability in case we get extremely small values in v_t:

AdamW 是有状态的：对于每个参数，它跟踪其一阶矩和二阶矩的运行估计。因此，AdamW 使用额外的内存来换取改进的稳定性和收敛性。除了学习率 η 之外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2)，以及权重衰减率 λ。典型应用将 (β_1, β_2) 设置为 $(0.9, 0.999)$，但像 LLaMA [H. Touvron et al., 2023] 和 GPT-3 [T. B. Brown et al., 2020] 这样的大型语言模型通常用 $(0.9, 0.95)$ 训练。该算法可以写成如下形式，其中 ϵ 是一个小值（例如 10^{-8}），用于在 v_t 中获得极小值时提高数值稳定性：

```
Algorithm 1: AdamW Optimizer / 算法 1: AdamW 优化器

1  init(theta_0)  Initialize learnable parameters / 初始化可学习参数
2  m_0 <- 0  Initial value of the first moment vector; same shape as theta / 一阶矩向量的初始值；形状与 theta 相同
3  v_0 <- 0  Initial value of the second moment vector; same shape as theta / 二阶矩向量的初始值；形状与 theta 相同
4  for t = 1, ..., T do
5      Sample batch of data B_t / 采样数据批次 B_t
6      g_t <- nabla L(B_t; theta_{t-1})  Compute the gradient of the loss / 计算损失的梯度
7      alpha_t <- eta * (1 - beta_2^t) / (1 - beta_1^t)  Compute adjusted eta for iteration t / 为第 t 次迭代计算调整后的 eta
8      theta_t <- theta_{t-1} - lambda * eta * theta_{t-1}  Apply weight decay / 应用权重衰减
9      m_t <- beta_1 * m_{t-1} + (1 - beta_1) * g_t  Update the first moment estimate / 更新一阶矩估计
10     v_t <- beta_2 * v_{t-1} + (1 - beta_2) * g_t^2  Update the second moment estimate / 更新二阶矩估计
11     theta_t <- theta_t - alpha_t * m_t / (sqrt(v_t) + epsilon)  Apply moment-adjusted weight updates / 应用矩调整的权重更新
12 end for
```

Note that t starts at 1. You will now implement this optimizer.

注意 t 从 1 开始。现在你将实现这个优化器。

Problem (adamw): Implement AdamW (2 points)

Deliverable: Implement the AdamW optimizer as a subclass of `torch.optim.Optimizer`. Your class should take the learning rate η in `__init__`, as well as the β_1, β_2 and λ hyperparameters. To help you keep state, the base `Optimizer` class gives you a dictionary `self.state`, which maps `nn.Parameter` objects to a dictionary that stores any information you need for that parameter (for AdamW, this would be the moment estimates). Implement `adapters.get_adamw_cls` and make sure it passes `uv run pytest -k test_adamw`.

问题（adamw）：实现 AdamW（2 分）

交付物：将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应在 `__init__` 中接受学习率 η，以及 β_1、β_2 和 λ 超参数。为帮助你保持状态，基类 `Optimizer` 为你提供了一个字典

`self.state`，它将 `nn.Parameter` 对象映射到一个存储你为该参数所需的任何信息的字典（对于 AdamW，这将是矩估计）。实现 `adapters.get_adamw_cls` 并确保它通过 `uv run pytest -k test_adamw`。

Problem (adamw_accounting): Resource accounting for training with AdamW (2 points)

Let us compute how much memory and compute running AdamW requires. Assume we are using float32 for every tensor.

(a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of the `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume $d_{ff} = \frac{8}{3} \times d_{model}$.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block
- final RMSNorm
- output embedding
- cross-entropy on logits

Deliverable: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the `batch_size`. What is the maximum batch size you can use and still fit within 80GB memory?

Deliverable: An expression that looks like $A \cdot batch\_size + B$ for numerical values A, B, and a number representing the maximum batch size.

(c) How many FLOPs does running one step of AdamW take?

Deliverable: An algebraic expression, with a brief justification.

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware's theoretical peak FLOP throughput [A. Chowdhery et al., 2022]. An NVIDIA H100 GPU has a theoretical peak of 495 teraFLOP/s for "float32" (actually TensorFloat-32, which in reality is "bfloat19") operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single H100? Following J. Kaplan et al. [25] and J. Hoffmann et al. [26], assume that the backward pass has twice the FLOPs of the forward pass.

Deliverable: The number of hours training would take, with a brief justification.

问题（adamw_accounting）：使用 AdamW 训练的资源核算（2 分）

让我们计算运行 AdamW 需要多少内存和计算。假设我们对每个张量使用 float32。

运行 AdamW 需要多少峰值内存？根据参数、激活值、梯度和优化器状态的内存使用量分解你的答案。用 `batch_size` 和模型超参数（`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`）表示你的答案。假设 $d_{ff} = \frac{8}{3} \times d_{model}$。

为简单起见，在计算激活值的内存使用量时，仅考虑以下组件：

- Transformer 块
  - RMSNorm(s)
  - Multi-head self-attention sublayer: QKV projections, QK^T matrix multiply, softmax, weighted sum of values, output projection. 多头自注意力子层：QKV 投影、QK^T 矩阵乘法、softmax、值的加权和、输出投影。
  - Position-wise feed-forward (SwiGLU): W_1, W_2, SiLU on the gate branch, element-wise product, W_3 逐位置前馈（SwiGLU）：W_1、W_2、门分支上的 SiLU、逐元素乘积、W_3
- 最终的 RMSNorm
- 输出嵌入
- 对 logits 的交叉熵

交付物：参数、激活值、梯度和优化器状态中每一项的代数表达式，以及总计。

将你的答案实例化为 GPT-2 XL 形状的模型，以获得仅依赖于 `batch_size` 的表达式。你可以使用并在 80GB 内存内仍能容纳的最大批次大小是多少？

交付物：形如 $A \cdot batch\_size + B$ 的表达式，其中 A、B 为数值，以及表示最大批次大小的数字。

运行一步 AdamW 需要多少 FLOPs?

交付物：一个代数表达式，附简短证明。

模型 FLOPs 利用率（MFU）定义为观察到的吞吐量（每秒 token 数）与硬件理论峰值 FLOP 吞吐量的比率 [A. Chowdhery et al., 2022]。NVIDIA H100 GPU 对 “float32”（实际上是 TensorFloat-32，实际上为 “bfloat19”）操作的理论峰值为 495 teraFLOP/s。假设你能够达到 50% MFU，在单个 H100 上以批次大小 1024 训练 GPT-2 XL 40 万步需要多长时间？遵循 J. Kaplan et al. [25] 和 J. Hoffmann et al. [26]，假设反向传播的 FLOPs 是前向传播的两倍。

交付物：训练所需的小时数，附简短证明。

4.4 Learning rate scheduling

学习率调度

The value for the learning rate that leads to the quickest decrease in loss often varies during training. In training Transformers, it is typical to use a learning rate schedule, where we start with a bigger learning rate, making quicker updates in the beginning, and slowly decay it to a smaller value as the model trains. In this assignment, we will implement the cosine annealing schedule used to train LLaMA [H. Touvron et al., 2023].

> 导致损失最快下降的学习率值通常在训练过程中变化。在训练 Transformer 时，通常使用学习率调度，我们从较大的学习率开始，在开始时进行更快的更新，并随着模型训练逐渐衰减到较小的值。在本作业中，我们将实现用于训练 LLaMA [H. Touvron et al., 2023] 的余弦退火调度。

A scheduler is simply a function that takes the current step t and other relevant parameters (such as the initial and final learning rates), and returns the learning rate to use for the gradient update at step t. The simplest schedule is the constant function, which will return the same learning rate given any t.

调度器只是一个函数，它接受当前步数 t 和其他相关参数（如初始和最终学习率），并返回在第 t 步梯度更新时使用的学习率。最简单的调度是常数函数，它对任意 t 返回相同的学习率。

The cosine annealing learning rate schedule takes (i) the current iteration t, (ii) the maximum learning rate η_{max}, (iii) the minimum (final) learning rate η_{min}, (iv) the number of warm-up iterations T_{warmup}, and (v) the final iteration of cosine annealing T_{cos}. The learning rate at iteration t is defined as:

余弦退火学习率调度接受 (i) 当前迭代 t，(ii) 最大学习率 η_{max}，(iii) 最小（最终）学习率 η_{min}，(iv) 预热迭代次数 T_{warmup}，和 (v) 余弦退火的最终迭代 T_{cos}。在第 t 次迭代的学习率定义为：

- (Warm-up) If $t < T_{warmup}$, then $\eta_t = \frac{t}{T_{warmup}} \eta_{max}$.

  （预热）如果 $t < T_{warmup}$，则 $\eta_t = \frac{t}{T_{warmup}} \eta_{max}$。

- (Cosine annealing) If $T_{warmup} \le t \le T_{cos}$, then $\eta_t = \eta_{min} + \frac{1}{2}\left(1 + \cos\left(\pi \frac{t - T_{warmup}}{T_{cos} - T_{warmup}}\right)\right)\left(\eta_{max} - \eta_{min}\right)$.

  （余弦退火）如果 $T_{warmup} \le t \le T_{cos}$，则 $\eta_t = \eta_{min} + \frac{1}{2}\left(1 + \cos\left(\pi \frac{t - T_{warmup}}{T_{cos} - T_{warmup}}\right)\right)\left(\eta_{max} - \eta_{min}\right)$。

- (Post-annealing) If $t > T_{cos}$, then $\eta_t = \eta_{min}$.

  （退火后）如果 $t > T_{cos}$，则 $\eta_t = \eta_{min}$。

Problem (learning_rate_schedule): Implement cosine learning rate schedule with warmup (1 point)

Write a function that takes t, η_{max}, η_{min}, T_{warmup} and T_{cos}, and returns the learning rate η_t according to the scheduler defined above. Then implement `adapters.get_lr_cosine_schedule` and make sure it passes `uv run pytest -k test_get_lr_cosine_schedule`.

问题（learning_rate_schedule）：实现带预热的余弦学习率调度（1 分）

编写一个函数，接受 t、η_{max}、η_{min}、T_{warmup} 和 T_{cos}，并根据上面定义的调度器返回学习率 η_t。然后实现 `adapters.get_lr_cosine_schedule` 并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 Gradient clipping

梯度裁剪

During training, we can sometimes hit training examples that yield large gradients, which can destabilize training. To mitigate this, one technique often employed in practice is gradient clipping. The idea is to enforce a limit on the norm of the gradient after each backward pass before taking an optimizer step.

在训练过程中，我们有时会遇到产生大梯度的训练样本，这可能会破坏训练的稳定性。为了缓解这个问题，在实践中经常采用的一种技术是梯度裁剪。其思想是在每次反向传播之后、执行优化器步骤之前，对梯度的范数施加限制。

Given the gradient g (for all parameters), we compute its ℓ_2-norm $\|g\|_2$. If this norm is less than a maximum value C, then we leave g as is; otherwise, we scale g down by a factor of $\frac{C}{\|g\|_2+\epsilon}$ (where a small ϵ, like 10^{-6}, is added for numeric stability). Note that the resulting norm will be just under C.

给定梯度 g（针对所有参数），我们计算其 ℓ_2-范数 $\|g\|_2$。如果该范数小于最大值 C，则保持 g 不变；否则，我们将 g 按因子 $\frac{C}{\|g\|_2+\epsilon}$ 缩小（其中添加小值 ϵ，如 10^{-6}，以保证数值稳定性）。注意，结果范数将刚好在 C 以下。

Problem (gradient_clipping): Implement gradient clipping (1 point)

Write a function that implements gradient clipping. Your function should take a list of parameters and a maximum ℓ_2-norm. It should modify each parameter gradient in place. Use $\epsilon = 10^{-6}$ (the PyTorch default). Then, implement the adapter `adapters.run_gradient_clipping` and make sure it passes `uv run pytest -k test_gradient_clipping`.

问题（gradient_clipping）：实现梯度裁剪（1 分）

编写一个实现梯度裁剪的函数。你的函数应该接受一个参数列表和一个最大 ℓ_2-范数。它应该原地修改每个参数梯度。使用 $\epsilon = 10^{-6}$（PyTorch 默认值）。然后，实现适配器 `adapters.run_gradient_clipping` 并确保它通过 `uv run pytest -k test_gradient_clipping`。

---

5 Training loop

训练循环

We will now finally put together the major components we've built so far: the tokenized data, the model, and the optimizer.

现在，我们终于将迄今为止构建的主要组件组合在一起：分词后的数据、模型和优化器。

5.1 Data Loader

数据加载器

The tokenized data (e.g., that you prepared in tokenizer_experiments) is a single sequence of tokens $x = (x_1, \ldots, x_N)$. Even though the source data might consist of separate documents (e.g., different web pages, or source code files), a common practice is to concatenate all of those into a single sequence of tokens, adding a delimiter between them (such as the <|endoftext|> token).

分词后的数据（例如，你在 tokenizer_experiments 中准备的）是一个单一的 token 序列 $x = (x_1, \ldots, x_N)$。尽管源数据可能由单独的文档组成（例如，不同的网页或源代码文件），常见的做法是将所有这些连接成一个单一的 token 序列，在它们之间添加分隔符（例如 <|endoftext|> token）。

A data loader turns this into a stream of batches, where each batch consists of B sequences of length S, paired with the corresponding next tokens, also with length S. For example, for $B = 1$, $S = 3$, ([2, 3, 4], [3, 4, 5]) would be one potential batch.

数据加载器将其转换为批次的流，其中每个批次由 B 个长度为 S 的序列组成，配上相应的下一个 token，也是长度 S。例如，对于 $B = 1$、$S = 3$，([2, 3, 4], [3, 4, 5]) 将是一个可能的批次。

Loading data in this way simplifies training for a number of reasons. First, any $1 \le i \le N - S$ gives a valid training sequence, so sampling training sequences is trivial. Since all training sequences have the same length, there's no need to pad input sequences, which improves hardware utilization (also by increasing batch size B). Finally, we also don't need to load the full dataset to sample training data, making it easy to handle large datasets that might not otherwise fit in memory.

以这种方式加载数据简化了训练，原因有几个。首先，任何 $1 \le i \le N - S$ 都给出一个有效的训练序列，因此采样训练序列是很简单的。由于所有训练序列具有相同的长度，无需填充输入序列，这提高了硬件利用率（也通过增加批次大小 B）。最后，我们也不需要加载完整的数据集来采样训练数据，从而更容易处理可能无法放入内存的大型数据集。

Problem (data_loading): Implement data loading (2 points)

Deliverable: Write a function that takes a numpy array x (integer array with token IDs), a `batch_size`, a `context_length` and a PyTorch device string (e.g., `'cpu'` or `'cuda:0'`), and returns a pair of tensors: the sampled input sequences and the corresponding next-token targets. Both tensors should have shape `(batch_size, context_length)` containing token IDs, and both should be placed on the requested device. To test your implementation against our provided tests, you will first need to implement the test adapter at `adapters.run_get_batch`. Then, run `uv run pytest -k test_get_batch` to test your implementation.

问题（data_loading）：实现数据加载（2 分）

交付物：编写一个函数，接受一个 numpy 数组 x（包含 token ID 的整数数组）、`batch_size`、`context_length` 和一个 PyTorch 设备字符串（例如 `'cpu'` 或 `'cuda:0'`），并返回一对张量：采样的输入序列和相应的下一个 token 目标。两个张量都应具有形状 `(batch_size, context_length)`，包含 token ID，并且都应放置在请求的设备上。要根据我们提供的测试来测试你的实现，你首先需要实现 `adapters.run_get_batch` 处的测试适配器。然后运行 `uv run pytest -k test_get_batch` 来测试你的实现。

Low-Resource Tip: Data loading on CPU or Apple Silicon

低资源提示：在 CPU 或 Apple Silicon 上加载数据

If you are planning to train your LM on CPU or Apple Silicon, you need to move your data to the correct device (and similarly, you should use the same device for your model later on).
If you are on CPU, you can use the `'cpu'` device string, and on Apple Silicon (M* chips), you can use the `'mps'` device string.

如果你计划在 CPU 或 Apple Silicon 上训练你的 LM，你需要将数据移动到正确的设备（类似地，你也应该在后续为你的模型使用相同的设备）。

如果你在 CPU 上，你可以使用 `'cpu'` 设备字符串，在 Apple Silicon（M* 芯片）上，你可以使用 `'mps'` 设备字符串。

What if the dataset is too big to load into memory? We can use a Unix system call named `mmap` which maps a file on disk to virtual memory, and lazily loads the file contents when that memory location is accessed. Thus, you can "pretend" you have the entire dataset in memory. Numpy implements this through `np.memmap` (or the flag `mmap_mode='r'` to `np.load`, if you originally saved the array with `np.save`), which will return a numpy array-like object that loads the entries on-demand as you access them. When sampling from your dataset (i.e., a numpy array) during training, be sure to load the dataset in memory-mapped mode (via `np.memmap` or the flag `mmap_mode='r'` to `np.load`, depending on how you saved the array). Make sure you also specify a dtype that matches the array that you're loading. It may be helpful to explicitly verify that the memory-mapped data looks correct (e.g., doesn't contain values beyond the expected vocabulary size).

> 如果数据集太大无法加载到内存中怎么办？我们可以使用名为 `mmap` 的 Unix 系统调用，它将磁盘上的文件映射到虚拟内存，并在访问该内存位置时惰性地加载文件内容。因此，你可以”假装”整个数据集在内存中。Numpy 通过 `np.memmap`（或 `np.load` 的标志 `mmap_mode='r'`，如果你最初用 `np.save` 保存了数组）实现这一点，它将返回一个类似 numpy 数组的对象，在你访问条目时按需加载它们。在训练期间从你的数据集（即 numpy 数组）采样时，请确保以内存映射模式加载数据集（通过 `np.memmap` 或 `np.load` 的标志 `mmap_mode='r'`，取决于你如何保存数组）。确保你还指定了与你正在加载的数组匹配的 dtype。显式验证内存映射数据看起来正确（例如，不包含超出预期词汇表大小的值）可能会有所帮助。

5.2 Checkpointing

检查点

In addition to loading data, we will also need to save models as we train. When running jobs, we often want to be able to resume a training run that stopped midway through (e.g., due to your job timing out, machine failure, etc). Even when all goes well, we might also want to later have access to intermediate models (e.g., to study training dynamics post-hoc, take samples from models at different stages of training, etc).

> 除了加载数据，我们还需要在训练时保存模型。当运行作业时，我们经常希望能够恢复中途停止的训练运行（例如，由于作业超时、机器故障等）。即使一切顺利，我们可能也想要稍后访问中间模型（例如，事后研究训练动态、从不同训练阶段的模型中采样等）。

A checkpoint should have all the states that we need to resume training. We of course want to be able to restore model weights at a minimum. If using a stateful optimizer (such as AdamW), we will also need to save the optimizer's state (e.g., in the case of AdamW, the moment estimates). Finally, to resume the learning rate schedule, we will need to know the iteration number we stopped at. PyTorch makes it easy to save all of these: every `nn.Module` has a `state_dict()` method that returns a dictionary with all learnable weights; we can restore these weights later with the sister method `load_state_dict()`. The same goes for any `torch.optim.Optimizer`. Finally, `torch.save(obj, dest)` can dump an object (e.g., a dictionary containing tensors as some values, but also regular Python objects like integers) to a file (path) or file-like object, which can then be loaded back into memory with `torch.load(src)`.

检查点应该包含我们恢复训练所需的所有状态。我们当然希望能够至少恢复模型权重。如果使用有状态优化器（如 AdamW），我们还需要保存优化器的状态（例如，对于 AdamW，是矩估计）。最后，为了恢复学习率调度，我们需要知道我们停止时的迭代次数。PyTorch 使得保存所有这些变得容易：每个 `nn.Module` 都有一个 `state_dict()` 方法，返回包含所有可学习权重的字典；我们可以稍后用姊妹方法 `load_state_dict()` 恢复这些权重。任何 `torch.optim.Optimizer` 也是如此。最后，`torch.save(obj, dest)` 可以将对象（例如，包含张量作为某些值的字典，但也包括常规 Python 对象如整数）转储到文件（路径）或类文件对象，然后可以用 `torch.load(src)` 将其加载回内存。

Problem (checkpointing): Implement model checkpointing (1 point)

Implement the following two functions to load and save checkpoints:

- `save_checkpoint(model, optimizer, iteration, out)` should dump all the state from the model, optimizer and iteration into the file-like object `out`. You can use the `state_dict` method of both the model and the optimizer to get their relevant states and use `torch.save(obj, out)` to dump `obj` into `out` (PyTorch supports either a path or a file-like object here). A typical choice is to have `obj` be a dictionary, but you can use whatever format you want as long as you can load your checkpoint later.

  This function expects the following parameters:

  - `model: torch.nn.Module`
  - `optimizer: torch.optim.Optimizer`
  - `iteration: int`
  - `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

- `load_checkpoint(src, model, optimizer)` should load a checkpoint from `src` (path or file-like object), and then recover the model and optimizer states from that checkpoint. Your function should return the iteration number that was saved to the checkpoint. You can use `torch.load(src)` to recover what you saved in your `save_checkpoint` implementation, and the `load_state_dict` method in both the model and optimizer to return them to their previous states.

  This function expects the following parameters:

  - `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

Implement the `adapters.run_save_checkpoint` and `adapters.run_load_checkpoint` adapters, and make sure they pass `uv run pytest -k test_checkpointing`.

问题（checkpointing）：实现模型检查点（1 分）

实现以下两个函数来加载和保存检查点：

- 应该将模型、优化器和迭代次数的所有状态转储到类文件对象 `out` 中。你可以使用模型和优化器的 `state_dict` 方法来获取它们的相关状态，并使用 `torch.save(obj, out)` 将 `obj` 转储到 `out` 中（PyTorch 在此处支持路径或类文件对象）。典型选择是让 `obj` 成为一个字典，但你可以使用任何你想要的格式，只要稍后可以加载你的检查点。

  此函数需要以下参数：

- 应该从 `src`（路径或类文件对象）加载检查点，然后从该检查点恢复模型和优化器状态。你的函数应该返回保存到检查点的迭代次数。你可以使用 `torch.load(src)` 来恢复你在 `save_checkpoint` 实现中保存的内容，并使用模型和优化器中的 `load_state_dict` 方法将它们恢复到之前的状态。

  此函数需要以下参数：

实现 `adapters.run_save_checkpoint` 和 `adapters.run_load_checkpoint` 适配器，并确保它们通过 `uv run pytest -k test_checkpointing`。

5.3 Training loop

训练循环

Now, it's finally time to put all of the components you implemented together into your main training script. It will pay off to make it easy to start training runs with different hyperparameters (e.g., by taking them as command-line arguments), since you will be doing these many times later to study how different choices impact training.

现在，终于到了将你实现的所有组件组合到主训练脚本中的时候了。花时间使其能够轻松地使用不同超参数启动训练运行将是值得的（例如，通过将它们作为命令行参数接受），因为你稍后将会多次这样做来研究不同选择如何影响训练。

Problem (training_together): Put it together (4 points)

Deliverable: Write a script that runs a training loop to train your model on user-provided input. In particular, we recommend that your training script allow for (at least) the following:

- Ability to configure and control the various model and optimizer hyperparameters.
- Memory-efficient loading of large training and validation datasets with `np.memmap`.

- Serializing checkpoints to a user-provided path.
- Periodically logging training and validation performance (e.g., to console and/or an external service like Weights and Biases).

问题（training_together）：组合起来（4 分）

交付物：编写一个运行训练循环的脚本，在用户提供的输入上训练你的模型。特别地，我们建议你的训练脚本（至少）允许以下内容：

- 能够配置和控制各种模型和优化器超参数。
- 使用 `np.memmap` 内存高效地加载大型训练和验证数据集。
- 将检查点序列化到用户提供的路径。
- 定期记录训练和验证性能（例如，输出到控制台和/或外部服务如 Weights and Biases）。

---

6 Generating text

生成文本

Now that we can train models, the last piece we need is the ability to generate text from our model. Recall that a language model takes in a (possibly batched) integer sequence of length `sequence_length` and produces a matrix of size `(sequence_length, vocab_size)`, where each element of the sequence is a probability distribution predicting the next token after that position. We will now write a few functions to turn this into a sampling scheme for new sequences.

现在我们可以训练模型了，最后一块我们需要的是从模型中生成文本的能力。回顾一下，语言模型接受一个长度为 `sequence_length` 的（可能批量的）整数序列，并产生大小为 `(sequence_length, vocab_size)` 的矩阵，其中序列的每个元素是一个预测该位置之后下一个 token 的概率分布。现在我们将编写一些函数，将其转换为新序列的采样方案。

Softmax

By standard convention, the language model output is the output of the final linear layer (the "logits") and so we have to turn this into a normalized probability via the softmax operation, which we saw earlier in Equation 10.

按照标准惯例，语言模型输出是最终线性层的输出（“logits”），因此我们必须通过 softmax 操作将其转换为归一化概率，我们之前在方程 10 中看到过。

Decoding

解码

To generate text (decode) from our model, we will provide the model with a sequence of prefix tokens (the “prompt”), and ask it to produce a probability distribution over the vocabulary that predicts the next token in the sequence. Then, we will sample from this distribution over the vocabulary items to determine the next output token.

要从我们的模型生成文本（解码），我们将为模型提供一个前缀 token 序列（“提示”），并要求它产生词汇表上的概率分布，预测序列中的下一个 token。然后，我们将从这个词汇表项的分布中采样，以确定下一个输出 token。

Concretely, one step of the decoding process should take in a sequence $x_{1\ldots t}$ and return a token x_{t+1} via the following equation,

具体来说，解码过程的一步应该接受一个序列 $x_{1\ldots t}$ 并通过以下方程返回一个 token x_{t+1}，

$$p\left(x_{t+1}=w \mid x_{1\ldots t}\right)=\frac{\exp\left(\ell_t[w]\right)}{\sum_{w'}\exp\left(\ell_t[w']\right)}$$

$$\ell_t=\text{TransformerLM}\left(x_{1\ldots t}\right)[-1]\in\mathbb{R}^{vocab\_size}$$

where TransformerLM is our model which takes as input a sequence of length sequence_length and produces a matrix of size (sequence_length, vocab_size), and we take the last element of this matrix, as we are looking for the next token prediction at the t-th position.

其中 TransformerLM 是我们的模型，它接受长度为 sequence_length 的序列作为输入，并产生大小为 (sequence_length, vocab_size) 的矩阵，我们取这个矩阵的最后一个元素，因为我们要查找第 t 个位置的下一个 token 预测。

This gives us a basic decoder by repeatedly sampling from these one-step conditionals (appending our previously-generated output token to the input of the next decoding timestep) until we generate the end-of-sequence token `<|endoftext|>` (or a user-specified maximum number of tokens to generate).

这给了我们一个基本的解码器，通过反复从这些单步条件分布中采样（将我们之前生成的输出 token 附加到下一个解码时间步的输入），直到我们生成序列结束 token `<|endoftext|>`（或用户指定的最大生成 token 数）。

Decoder tricks

解码器技巧

We will be experimenting with small models, and small models can sometimes generate very low-quality texts. Two simple decoder tricks can help fix these issues.

> 我们将使用小型模型进行实验，小型模型有时可能生成非常低质量的文本。两个简单的解码器技巧可以帮助解决这些问题。

First, in **temperature scaling** we modify our softmax with a temperature parameter τ, where the new softmax is

第一，在**温度缩放**中，我们用温度参数 τ 修改我们的 softmax，其中新的 softmax 为

$$\text{softmax}(\ell, \tau)_i = \frac{\exp(\ell_i / \tau)}{\sum_{j=1}^{vocab\_size} \exp(\ell_j / \tau)}.$$

Note how setting $\tau \to 0$ makes it so that the largest element of ℓ dominates, and the output of the softmax becomes a one-hot vector concentrated at this maximal element.

注意，设置 $\tau \to 0$ 会使得 ℓ 的最大元素占主导地位，softmax 的输出变为集中在这个最大元素处的 one-hot 向量。

Second, another trick is **nucleus** or **top-p sampling**, where we modify the sampling distribution by truncating low-probability tokens. Let p be a probability distribution that we get from a (temperature-scaled) softmax of size vocab_size. Nucleus sampling with hyperparameter p produces the next token according to the equation

第二，另一个技巧是**核采样**或 **top-p 采样**，我们通过截断低概率 token 来修改采样分布。令 p 是我们从大小为 vocab_size 的（温度缩放的）softmax 获得的概率分布。具有超参数 p 的核采样根据以下方程产生下一个 token：

$$p(x_{t+1} = w \mid x_t) = \begin{cases} \dfrac{p(w)}{\sum_{w' \in V^{(p)}} p(w')} & \text{if } w \in V^{(p)} \\ 0 & \text{otherwise} \end{cases}$$

where $V^{(p)}$ is the smallest set of indices such that $\sum_{w \in V^{(p)}} p(w) \geq p$. You can compute this quantity easily by first sorting the probability distribution p by magnitude, and selecting the largest vocabulary elements until you reach the target level of p.

其中 $V^{(p)}$ 是最小的索引集合，使得 $\sum_{w \in V^{(p)}} p(w) \geq p$。你可以通过首先按大小对概率分布 p 进行排序，然后选择最大的词汇表元素，直到达到目标 p 水平，来轻松计算此量。

Problem (decoding): Decoding (3 points)

Deliverable: Implement a function to decode from your language model. We recommend that you support the following features:

- Generate completions for a user-provided prompt (i.e., take in some $x_{1...t}$ and sample a completion until you hit an <|endoftext|> token).
- Allow the user to control the maximum number of generated tokens.
- Given a desired temperature value, apply softmax temperature scaling to the predicted next-token distributions before sampling.
- Top-p sampling ([A. Holtzman et al., 2020] also referred to as nucleus sampling), given a user-specified threshold value.

问题（decoding）：解码（3 分）

交付物：实现一个从你的语言模型中解码的函数。我们建议你支持以下功能：

- 为用户提供的提示生成补全（即，接受一些 $x_{1...t}$ 并采样补全，直到遇到 <|endoftext|> token）。
- 允许用户控制生成的最大 token 数。
- 给定所需的温度值，在采样之前对预测的下一个 token 分布应用 softmax 温度缩放。
- Top-p 采样（[A. Holtzman et al., 2020] 也称为核采样），给定用户指定的阈值。

---

7 Experiments

实验

Now it is time to put everything together and train (small) language models on a pretraining dataset.

现在是时候将所有内容组合在一起，在预训练数据集上训练（小型）语言模型了。

7.1 How to Run Experiments and Deliverables

如何运行实验和交付物

The best way to understand the rationale behind the architectural components of a Transformer is to actually modify it and run it yourself. There is no substitute for hands-on experience.

理解 Transformer 架构组件背后原理的最佳方式是实际修改它并自己运行。没有什么能替代动手经验。

To this end, it's important to be able to experiment quickly, consistently, and keep records of what you did. To experiment quickly, we will be running many experiments on a small-scale model (about 17M total parameters) and simple dataset (TinyStories). To do things consistently, you will ablate components and vary hyperparameters in a systematic way, and to keep records we will ask you to submit a log of your experiments and learning curves associated with each experiment.

为此，能够快速、一致地进行实验并保持记录是很重要的。为了快速实验，我们将在小规模模型（约 17M 总参数）和简单数据集（TinyStories）上运行许多实验。为了做事一致，你将以系统化的方式消融组件和改变超参数，为了保持记录，我们将要求你提交实验日志和与每个实验相关的学习曲线。

To make it possible to submit loss curves, make sure to periodically evaluate validation losses and record both the number of steps and wall-clock times. You might find logging infrastructure such as Weights and Biases helpful.

为了能够提交损失曲线，请确保定期评估验证损失并记录步数和实际运行时间。你可能会发现像 Weights and Biases 这样的日志基础设施很有帮助。

Problem (experiment_log): Experiment logging (3 points)

For your training and evaluation code, create experiment tracking infrastructure that allows you to track your experiments and loss curves with respect to gradient steps and wall-clock time.

Deliverable: Logging infrastructure code for your experiments and an experiment log (a document of all the things you tried) for the assignment problems below in this section.

问题（experiment_log）：实验日志（3 分）

为你的训练和评估代码创建实验跟踪基础设施，使你能够跟踪实验和相对于梯度步数和实际运行时间的损失曲线。

交付物：你的实验日志基础设施代码和针对本节下面作业问题的实验日志（记录你尝试的所有内容的文档）。

7.2 TinyStories

We are going to start with a very simple dataset (TinyStories; R. Eldan et al. [1]) where models will train quickly, and we can see some interesting behaviors. The instructions for getting this dataset are in Section 1. An example of what this dataset looks like is below.

我们将从一个非常简单的数据集（TinyStories; R. Eldan et al. [1]）开始，在这个数据集上模型训练得很快，我们可以看到一些有趣的行为。获取此数据集的说明在第 1 节。以下是该数据集的一个示例。

Example (tinystories_example): One example from TinyStories

示例（tinystories_example）：TinyStories 中的一个例子

Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, “Wow, that is a really amazing vase! Can I buy it?” The shopkeeper smiled and said, “Of course you can. You can take it home and show all your friends how amazing it is!” So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn’t believe how lucky Ben was. And that’s how Ben found an amazing vase in the store!

从前有一个叫 Ben 的小男孩。Ben 喜欢探索周围的世界。他看到了许多令人惊奇的事物，比如商店里陈列的美丽花瓶。一天，Ben 走过商店时发现了一个非常特别的花瓶。当 Ben 看到它时，他惊讶极了！他说：“哇，那真是一个了不起的花瓶！我能买它吗？”店主微笑着说：“当然可以。你可以把它带回家，向所有朋友展示它有多棒！”于是 Ben 把花瓶带回家，他非常自豪！他把朋友们叫过来，向他们展示这个了不起的花瓶。他的所有朋友都觉得这个花瓶很漂亮，不敢相信 Ben 有多么幸运。这就是 Ben 如何在商店里发现了一个了不起的花瓶！

7.2.1 Hyperparameter tuning

超参数调优

We will tell you some very basic hyperparameters to start with and ask you to find some settings for others that work well.

我们将告诉你一些非常基本的超参数作为起点，并要求你为其他参数找到效果良好的设置。

- **Vocab size** 10000. Typical vocabulary sizes are in the tens to hundreds of thousands. You should vary this and see how the vocabulary and model behavior change.

  词汇表大小 10000。典型的词汇表大小在数万到数十万之间。你应该改变此值，看看词汇表和模型行为如何变化。

- **Context length** 256. Simple datasets such as TinyStories might not need long sequence lengths, but for the later OpenWebText data, you may want to vary this. Try varying this and seeing the impact on both the per-iteration runtime and the final perplexity.

  上下文长度 256。像 TinyStories 这样的简单数据集可能不需要长序列长度，但对于后面的 OpenWebText 数据，你可能想要改变此值。尝试改变此值，看看对每次迭代运行时间和最终困惑度的影响。

- **d_model** 512. This is slightly smaller than the 768 dimensions used in many small Transformer papers, but this will make things faster.

> **d_model** 512。这比许多小型 Transformer 论文中使用的 768 维度略小，但这会使事情更快。

- **d_ff** 1344. This is roughly $\frac{8}{3}d_{model}$ while being a multiple of 64, which is good for GPU performance.

> **d_ff** 1344。这大约是 $\frac{8}{3}d_{model}$，同时是 64 的倍数，这对 GPU 性能有好处。

- **RoPE theta parameter** 10000.
- **Number of layers and heads** 4 layers, 16 heads. Together, this will give about 17M non-embedding parameters which is a fairly small Transformer.

> **层数和头数** 4 层，16 头。加起来，这将给出约 17M 非嵌入参数，是一个相当小的 Transformer。

- **Total tokens processed** 327,680,000 (your batch size × total step count × context length should equal roughly this value).

> **处理的总 token 数** 327,680,000（你的批次大小 × 总步数 × 上下文长度应大约等于此值）。

You should do some trial and error to find good defaults for the following other hyperparameters:

> 你应该通过试错法为以下其他超参数找到良好的默认值：

learning rate, learning rate warmup, other AdamW hyperparameters (β_1, β_2, ϵ), and weight decay. You can find some typical choices of such hyperparameters in D. P. Kingma et al. [22].

学习率、学习率预热、其他 AdamW 超参数（β_1、β_2、ϵ）和权重衰减。你可以在 D. P. Kingma et al. [22] 中找到这些超参数的一些典型选择。

7.2.2 Putting it together

组合起来

Now you can put everything together by getting a trained BPE tokenizer, tokenizing the training dataset, and running this in the training loop that you wrote. **Important note:** If your implementation is correct and efficient, the above hyperparameters should result in a roughly 20–30 minute runtime on 1 B200 GPU. If you have runtimes that are much longer, please check and make sure your dataloading, checkpointing, or validation loss code is not bottlenecking your runtimes and that your implementation is properly batched.

> 现在你可以通过获取训练好的 BPE 分词器、对训练数据集进行分词，并在你编写的训练循环中运行它来将所有内容组合起来。**重要提示：**如果你的实现正确且高效，上述超参数应该会在 1 个 B200 GPU 上产生大约 20-30 分钟的运行时间。如果你的运行时间长得很多，请检查并确保你的数据加载、检查点或验证损失代码没有成为运行时间的瓶颈，并且你的实现是适当批处理的。

7.2.3 Tips and tricks for debugging model architectures

调试模型架构的技巧和窍门

We highly recommend getting comfortable with your IDE's built-in debugger (e.g., VSCode/Zed), which will save you time compared to debugging with print statements. If you use a text editor, you can use something like `ipdb`. A few other good practices when debugging model architectures are:

> 我们强烈建议你熟悉 IDE 的内置调试器（例如 VSCode/Zed），与使用 print 语句调试相比，这将节省你的时间。如果你使用文本编辑器，你可以使用像 `ipdb` 这样的工具。调试模型架构时的其他一些好做法是：

- A common first step when developing any neural net architecture is to overfit to a single minibatch. If your implementation is correct, you should be able to quickly drive the training loss to near-zero.

> 开发任何神经网络架构时的常见第一步是对单个小批次进行过拟合。如果你的实现正确，你应该能够迅速将训练损失降低到接近零。

- Set debug breakpoints in various model components, and inspect the shapes of intermediate tensors to make sure they match your expectations.

> 在各种模型组件中设置调试断点，并检查中间张量的形状以确保它们符合你的预期。

- Monitor the norms of activations, model weights, and gradients to make sure they are not exploding or vanishing.

> 监控激活值、模型权重和梯度的范数，确保它们没有爆炸或消失。

Problem (learning_rate): Tune the learning rate (2 B200 hrs) (3 points)

The learning rate is one of the most important hyperparameters to tune. Taking the base model you've trained, answer the following questions:

(a) Perform a hyperparameter sweep over the learning rates and report the final losses (or note divergence if the optimizer diverges).

Deliverable: Learning curves associated with multiple learning rates. Explain your hyperparameter search strategy.

Deliverable: A model with validation loss (per-token) on TinyStories of at most 1.45.

(b) Folk wisdom is that the best learning rate is "at the edge of stability." Investigate how the point at which learning rates diverge is related to your best learning rate.

Deliverable: Learning curves of increasing learning rate which include at least one divergent run and an analysis of how this relates to convergence rates.

问题（learning_rate）：调节学习率（2 B200 小时）（3 分）

学习率是需要调节的最重要的超参数之一。使用你训练的基础模型，回答以下问题：

对学习率执行超参数搜索，并报告最终损失（或如果优化器发散则注明发散）。

交付物：与多个学习率相关的学习曲线。解释你的超参数搜索策略。

交付物：在 TinyStories 上具有每 token 验证损失至多 1.45 的模型。

Low-Resource Tip: Train for a few steps on CPU or Apple Silicon 低资源提示：在 CPU 或 Apple Silicon 上训练几步

If you are running on cpu or mps, you should instead reduce the total tokens processed count to 40,000,000, which will be sufficient to produce reasonably fluent text. You may also increase the target validation loss from 1.45 to 2.00.

如果你在 cpu 或 mps 上运行，你应将处理的总 token 数减少到 40,000,000，这足以产生相当流利的文本。你也可以将目标验证损失从 1.45 增加到 2.00。

Running our solution code with a tuned learning rate on an M4 Max chip and 36 GB of RAM, we use batch size × total step count × context length = 32 × 5000 × 256 = 40,960,000 tokens, which takes 1 hour and 22 minutes on cpu and 36 minutes on mps. At step 5000, we achieve a validation loss of 1.80.

在 M4 Max 芯片和 36 GB RAM 上使用经过调节的学习率运行我们的解决方案代码，我们使用批次大小 × 总步数 × 上下文长度 = 32 × 5000 × 256 = 40,960,000 token，在 cpu 上需要 1 小时 22 分钟，在 mps 上需要 36 分钟。在第 5000 步，我们达到了 1.80 的验证损失。

Some additional tips: 一些额外的提示： - When using T training steps, we suggest adjusting the cosine learning rate decay schedule to terminate its decay (i.e., reach the minimum learning rate) at precisely step T. 当使用 T 个训练步数时，我们建议调整余弦学习率衰减调度，使其在精确的第 T 步终止衰减（即达到最小学习率）。 - When using mps, do not use TF32 kernels, i.e., do not set `torch.set_float32_matmul_precision('high')` as you might with cuda devices. 当使用 mps 时，不要使用 TF32 内核，即不要像使用 cuda 设备那样设置 `torch.set_float32_matmul_precision('high')`。 - You can speed up training by JIT-compiling your model with `torch.compile`. 你可以通过使用 `torch.compile` 进行 JIT 编译来加速训练。 - On cpu, compile your model with `model = torch.compile(model)` 在 cpu 上，使用 `model = torch.compile(model)` 编译你的模型 - On mps, you can somewhat optimize the backward pass using `model = torch.compile(model, backend="aot_eager")` 在 mps 上，你可以使用 `model = torch.compile(model, backend="aot_eager")` 在一定程度上优化反向传播

民间智慧认为，最好的学习率是”处于稳定边缘”。研究学习率发散的点与你最佳学习率之间的关系。

交付物：递增学习率的学习曲线，包括至少一次发散运行，以及这与收敛速度如何相关的分析。

Now let's vary the batch size and see what happens to training. Batch sizes are important — they let us get higher efficiency from our GPUs by doing larger matrix multiplies, but is it true that we always want batch sizes to be large? Let's run some experiments to find out.

现在让我们改变批次大小，看看训练会发生什么。批次大小很重要——它们让我们通过执行更大的矩阵乘法从 GPU 获得更高的效率，但是我们是否总是希望批次大小很大呢？让我们运行一些实验来找出答案。

Problem (batch_size_experiment): Batch size variations (1 B200 hr) (1 point)

Vary your batch size all the way from 1 to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like 64 and 128.

Deliverable: Learning curves for runs with different batch sizes. The learning rates should be optimized again if necessary.

Deliverable: A few sentences discussing your findings on batch sizes and their impacts on training.

问题（batch_size_experiment）：批次大小变化（1 B200 小时）（1 分）

将你的批次大小从 1 一直变化到 GPU 内存限制。至少尝试中间的几种批次大小，包括像 64 和 128 这样的典型大小。

交付物：不同批次大小运行的学习曲线。如有必要，应再次优化学习率。

交付物：几句讨论你关于批次大小及其对训练影响的发现。

With your decoder in hand, we can now generate text! We will generate from the model and see how good it is. As a reference, you should get outputs that look at least as good as the example below.

有了解码器，我们现在可以生成文本了！我们将从模型生成文本，看看它有多好。作为参考，你应该获得至少与以下示例一样好的输出。

Example (ts_generate_example): Sample output from a TinyStories language model

示例（ts_generate_example）：来自 TinyStories 语言模型的样例输出

Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

从前，有一个漂亮的女孩叫 Lily。她喜欢吃口香糖，特别是大大的黑黑的那种。一天，Lily 的妈妈让她帮忙做晚饭。Lily 非常兴奋！她喜欢帮妈妈的忙。Lily 的妈妈为晚餐做了一大锅汤。Lily 非常开心，

说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭后，Lily 的妈妈做了一些美味的汤。Lily 非常喜欢！她说：“谢谢你，妈妈！这汤太好喝了！”她的妈妈微笑着说：“我很高兴你喜欢，Lily。”她们做完了饭，继续一起做饭。故事结束。

Low-Resource Tip: Generate text on CPU or Apple Silicon

低资源提示：在 CPU 或 Apple Silicon 上生成文本

If instead you used the low-resource configuration with 40M tokens processed, you should see generations that still resemble English but are not as fluent as above.

如果你改为使用处理 40M token 的低资源配置，你应该看到生成的内容仍然类似英语，但不如上述内容流利。

Here is the precise problem statement and what we ask for:

以下是精确的问题陈述和我们要求的内容：

Problem (generate): Generate text (1 point)

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters (temperature, top-p, etc.) to get fluent outputs.

Deliverable: Text dump of at least 256 tokens of text (or until the first `<|endoftext|>` token), and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

问题（generate）：生成文本（1 分）

使用你的解码器和训练好的检查点，报告你的模型生成的文本。你可能需要调整解码器参数（温度、top-p 等）以获得流利的输出。

交付物：至少 256 个 token 的文本转储（或直到第一个 `<|endoftext|>` token），以及对输出的流利度和至少两个影响输出好坏的因素的简短评论。

7.3 Ablations and architecture modification

消融实验和架构修改

The best way to understand the Transformer is to actually modify it and see how it behaves. We will now do a few simple ablations and modifications.

理解 Transformer 的最佳方式是实际修改它并观察其行为。现在我们将进行一些简单的消融和修改。

Ablation 1: layer normalization

消融 1：层归一化

It is often said that layer normalization is important for the stability of Transformer training. But perhaps we want to live dangerously. Let's remove RMSNorm from each of our Transformer blocks and see what happens.

人们常说层归一化对 Transformer 训练的稳定性很重要。但也许我们想冒险试试。让我们从每个 Transformer 块中移除 RMSNorm，看看会发生什么。

Problem (layer_norm_ablation): Remove RMSNorm and train (0.5 B200 hrs) (1 point)

Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

Deliverable: A learning curve for when you remove RMSNorms and train, as well as a learning curve for the best learning rate.

Deliverable: A few sentences of commentary on the impact of RMSNorm.

问题（layer_norm_ablation）：移除 RMSNorm 并训练（0.5 B200 小时）（1 分）

从你的 Transformer 中移除所有 RMSNorm 并训练。在之前的最优学习率下会发生什么？你能通过使用较低的学习率来获得稳定性吗？

交付物：移除 RMSNorm 并训练的学习曲线，以及最佳学习率下的学习曲线。

交付物：几句关于 RMSNorm 影响的评论。

Let's now investigate another layer normalization choice that seems arbitrary at first glance. Pre-norm Transformer blocks are defined as

现在让我们研究另一个初看似乎随意的层归一化选择。Pre-norm Transformer 块定义为

$$y = x + \mathrm{MultiHeadSelfAttention}\left(\mathrm{RMSNorm}\left(x\right)\right)$$

$$y = y + \mathrm{FFN}\left(\mathrm{RMSNorm}\left(y\right)\right).$$

This is one of the few 'consensus' modifications to the original Transformer architecture, which used a post-norm approach as

这是对原始 Transformer 架构的少数”共识”修改之一，原始架构使用 post-norm 方法，即

$$y = \mathrm{RMSNorm}\left(x + \mathrm{MultiHeadSelfAttention}\left(x\right)\right)$$

$$y = \mathrm{RMSNorm}\left(y + \mathrm{FFN}\left(y\right)\right).$$

Let's revert back to the post-norm approach and see what happens.

让我们回退到 post-norm 方法，看看会发生什么。

Problem (pre_norm_ablation): Implement post-norm and train (0.5 B200 hrs) (1 point)

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens.

Deliverable: A learning curve for a post-norm Transformer, compared to the pre-norm one.

问题（pre_norm_ablation）：实现 post-norm 并训练（0.5 B200 小时）（1 分）

将你的 pre-norm Transformer 实现修改为 post-norm。使用 post-norm 模型训练，看看会发生什么。

交付物：post-norm Transformer 的学习曲线，与 pre-norm 的比较。

We see that layer normalization has a major impact on the behavior of the Transformer, and that even the position of the layer normalization is important.

我们看到层归一化对 Transformer 的行为有重大影响，甚至层归一化的位置也很重要。

Ablation 2: position embeddings

消融 2：位置嵌入

We will next investigate the impact of the position embeddings on the performance of the model. Specifically, we will compare our base model (with RoPE) with not including position embeddings at all (NoPE). It turns out that decoder-only transformers, i.e., those with a causal mask as we have implemented, can in theory infer relative or absolute position information without being provided with position embeddings explicitly [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]. We will now test empirically how NoPE performs compared to RoPE.

接下来我们将研究位置嵌入对模型性能的影响。具体来说，我们将比较我们的基础模型（带 RoPE）与完全不包含位置嵌入（NoPE）。事实证明，decoder-only Transformer，即那些具有我们已实现的因果掩码的 Transformer，理论上可以在不显式提供位置嵌入的情况下推断相对或绝对位置信息 [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。现在我们将通过实验测试 NoPE 与 RoPE 相比的表现如何。

Problem (no_pos_emb): Implement NoPE (0.5 B200 hrs) (1 point)

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

Deliverable: A learning curve comparing the performance of RoPE and NoPE.

问题（no_pos_emb）：实现 NoPE（0.5 B200 小时）（1 分）

修改你带 RoPE 的 Transformer 实现，完全移除位置嵌入信息，并观察发生了什么。

交付物：比较 RoPE 和 NoPE 性能的学习曲线。

Ablation 3: SwiGLU vs. SiLU

消融 3：SwiGLU vs. SiLU

Next, we will follow N. Shazeer [20] and test the importance of gating in the feed-forward network, by comparing the performance of SwiGLU feed-forward networks versus feed-forward networks using SiLU activations but no gated linear unit (GLU):

接下来，我们将遵循 N. Shazeer [20]，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但无门控线性单元（GLU）的前馈网络的性能，来测试门控在前馈网络中的重要性：

$$\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2 \,\mathrm{SiLU}(x W_1).$$

Recall that in our SwiGLU implementation, we set the dimensionality of the inner feed-forward layer to be roughly $d_{ff} = \frac{8}{3} d_{model}$ (while ensuring that $d_{ff} \bmod 64 = 0$, to make use of GPU tensor cores). In this ablation baseline, your $\mathrm{FFN}_{\mathrm{SiLU}}$ implementation should instead set $d_{ff} = 4 \times d_{model}$, to approximately match the parameter count of the default SwiGLU feed-forward network (which has three instead of two weight matrices).

回顾在我们的 SwiGLU 实现中，我们将内部前馈层的维度设置为大约 $d_{ff} = \frac{8}{3} d_{model}$（同时确保 $d_{ff} \bmod 64 = 0$，以利用 GPU 张量核心）。在这个消融基线中，你的 $\mathrm{FFN}_{\mathrm{SiLU}}$ 实现应该设置 $d_{ff} = 4 \times d_{model}$，以大致匹配默认 SwiGLU 前馈网络的参数数量（它有三个权重矩阵而不是两个）。

Problem (swiglu_ablation): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point)

Deliverable: A learning curve comparing the performance of SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

Deliverable: A few sentences discussing your findings.

问题（swiglu_ablation）：SwiGLU vs. SiLU（0.5 B200 小时）（1 分）

交付物：比较 SwiGLU 和 SiLU 前馈网络性能的学习曲线，参数数量大致匹配。

交付物：几句讨论你发现的句子。

Low-Resource Tip: Online students with limited GPU resources should test modifications on TinyStories

低资源提示：GPU 资源有限的在线学生应在 TinyStories 上测试修改

In the remainder of the assignment, we will move to a larger-scale, noisier web dataset (OpenWebText), experimenting with architecture modifications and (optionally) making a submission to the course leaderboard.

It takes a long time to train an LM to fluency on OpenWebText, so we suggest that online students with limited GPU access continue testing modifications on TinyStories (using validation loss as a metric to evaluate performance).

在作业的剩余部分，我们将转移到更大规模、更嘈杂的网络数据集（OpenWebText），尝试架构修改并（可选地）提交到课程排行榜。

在 OpenWebText 上训练一个 LM 到流利需要很长时间，因此我们建议 GPU 访问有限的在线学生继续在 TinyStories 上测试修改（使用验证损失作为评估性能的指标）。

7.4 Running on OpenWebText

在 OpenWebText 上运行

We will now move to a more standard pretraining dataset created from a web crawl. A small sample of OpenWebText [A. Gokaslan et al., 2019] is also provided as a single text file: see Section 1 for how to access this file.

现在我们将转移到从网页爬取创建的更标准的预训练数据集。OpenWebText [A. Gokaslan et al., 2019] 的一小部分样本也以单个文本文件提供：有关如何访问此文件，请参见第 1 节。

Here is an example from OpenWebText. Note how the text is much more realistic, complex, and varied. You may want to look through the training dataset to get a sense of what training data looks like for a web-scraped corpus.

以下是来自 OpenWebText 的一个例子。注意文本是多么更加真实、复杂和多样化。你可能想浏览训练数据集，以了解网页抓取语料库的训练数据是什么样的。

Example (owt_example): One example from OWT

示例（owt_example）：OWT 中的一个例子

Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in The Numbers Game, “no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players.” With a few clicks here and there, you can find out that Noah Syndergaard’s fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game’s highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball’s culture: the analytical hobbyist.

That empowerment comes with added scrutiny — on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site’s catching metrics needed to be reworked, and that it would take a learned mind — someone who could tackle complex statistical modeling problems — to complete the job.

“He freaks us out.” Harry Pavlidis

> Pavlidis had a hunch that Judge “got it” based on the latter’s writing and their interaction at a site-sponsored ballpark event. […]

Note: You may have to re-tune your hyperparameters such as learning rate or batch size for this experiment.

> 注意：你可能需要为此实验重新调节超参数，如学习率或批次大小。

Problem (main_experiment): Experiment on OWT (2 B200 hrs) (2 points)

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. How well does this model do?

Deliverable: A learning curve of your language model on OpenWebText. Describe the difference in losses from TinyStories — how should we interpret these losses?

Deliverable: Generated text from OpenWebText LM, in the same format as the TinyStories outputs. How is the fluency of this text? Why is the output quality worse even though we have the same model and compute budget as TinyStories?

问题（main_experiment）：在 OWT 上实验（2 B200 小时）（2 分）

使用与 TinyStories 相同的模型架构和总训练迭代次数，在 OpenWebText 上训练你的语言模型。这个模型表现如何?

交付物：你的语言模型在 OpenWebText 上的学习曲线。描述与 TinyStories 的损失差异——我们应该如何解释这些损失?

交付物：来自 OpenWebText LM 的生成文本，格式与 TinyStories 输出相同。这个文本的流利度如何？为什么即使我们拥有与 TinyStories 相同的模型和计算预算，输出质量却更差?

7.5 Your own modification + leaderboard

你自己的修改 + 排行榜

Congratulations on getting to this point. You’re almost done! You will now try to improve upon the Transformer architecture, and see how your hyperparameters and architecture stack up against other students in the class.

> 恭喜你到达这一步。你几乎完成了！现在你将尝试改进 Transformer 架构，并看看你的超参数和架构与班上其他同学相比如何。

Rules for the leaderboard

排行榜规则

There are no restrictions other than the following:

> 除了以下限制外，没有其他限制：

- **Runtime:** Your submission can run for at most 45 minutes on a B200. You might want to enforce this in your submission script if you use either SLURM or Modal.

> **运行时间：**你的提交在 B200 上最多运行 45 分钟。如果你使用 SLURM 或 Modal，你可能想在提交脚本中强制执行此限制。

- **Data:** You may only use the OpenWebText training dataset that we provide.

> **数据：**你只能使用我们提供的 OpenWebText 训练数据集。

- Otherwise, you are free to do whatever your heart desires.

> 除此之外，你可以自由地做任何你想做的事情。

If you are looking for some ideas on what to implement, you can check out some of these resources:

> 如果你在寻找一些可以实现的灵感，你可以查看以下资源：

- State-of-the-art open-source LLM families, such as Llama 3 [A. Grattafiori et al., 2024] or Qwen 2.5 [A. Yang et al., 2024].

> 最先进的开源 LLM 家族，如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。

- The NanoGPT speedrun repository (github.com/KellerJordan/modded-nanogpt), where community members post many interesting modifications for "speedrunning" small-scale language model pretraining. For example, a common modification that dates back to the original Transformer paper is to tie the weights of the input and output embeddings together (see A. Vaswani et al. [8] (Section 3.4) and A. Chowdhery et al. [16] (Section 2)). If you do try weight tying, you may have to decrease the standard deviation of the embedding/LM head init.

> NanoGPT speedrun 仓库（github.com/KellerJordan/modded-nanogpt），社区成员在其中发布了许多有趣的修改，用于"速通"小规模语言模型预训练。例如，一种可追溯到原始 Transformer 论文的常见修改是将输入和输出嵌入的权重绑定在一起（参见 A. Vaswani et al. [8]（第 3.4 节）和 A. Chowdhery et al. [16]（第 2 节））。如果你尝试权重绑定，你可能需要降低嵌入/LM 头初始化的标准差。

You will want to test these on either a small subset of OpenWebText or on TinyStories before trying the full 45-minute run.

> 在尝试完整的 45 分钟运行之前，你需要先在 OpenWebText 的小子集或 TinyStories 上测试这些。

As a caveat, we do note that some of the modifications you may find working well in this leaderboard may not generalize to larger-scale pretraining. We will explore this idea further in the scaling laws unit of the course.

需要注意的是，你在这个排行榜中发现效果良好的一些修改可能不会泛化到更大规模的预训练。我们将在课程的缩放定律单元中进一步探索这个想法。

Problem (leaderboard): Leaderboard (10 B200 hrs) (6 points)

You will train a model under the leaderboard rules above with the goal of minimizing the validation loss of your language model within 0.75 B200-hours.

Deliverable: The final validation loss that was recorded, an associated learning curve that clearly shows a wall-clock-time x-axis that is less than 45 minutes, and a description of what you did. We expect a leaderboard submission to beat at least the naïve baseline of a 5.0 loss. Submit to the leaderboard here: github.com/stanford-cs336/assignment1-basics-leaderboard.

问题（leaderboard）：排行榜（10 B200 小时）（6 分）

你将按照上述排行榜规则训练一个模型，目标是在 0.75 B200-小时内最小化你的语言模型的验证损失。

交付物：记录的最终验证损失、清楚显示实际运行时间 x 轴小于 45 分钟的相关学习曲线，以及你所做内容的描述。我们期望排行榜提交至少超过 5.0 损失的朴素基线。在此提交到排行榜：github.com/stanford-cs336/assignment1-basics-leaderboard。

Bibliography

参考文献

[1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.

[2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.

[3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.

[4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.

[5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.

[8] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.

[9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.

[10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.

[12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.

[13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.

[14] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: https://arxiv.org/abs/2407.21783

[15] A. Yang et al., “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.

[16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.

[17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.

[18] S. Elfwing, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: https://arxiv.org/abs/1702.03118

[19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: https://arxiv.org/abs/1612.08083

[20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

[21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.

[22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.

[23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.

[24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.

[25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.

[26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.

[27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.

[28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: 10.18653/v1/D19-1443.

[29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: https://openreview.net/forum?id=Drrl2gcjzl

Source Links

源文链接

The following external links appeared in the source PDF; page numbers refer to the source.

以下外部链接均见于源 PDF；页码指源文页码。

- Source p. 46

  源文第 46 页：https://arxiv.org/abs/1612.08083

- Source p. 46

  源文第 46 页：https://arxiv.org/abs/1702.03118

- Source p. 46

  源文第 46 页：https://arxiv.org/abs/2407.21783

- Source p. 35

  源文第 35 页：https://developer.apple.com/documentation/metalperformanceshaders

- Source p. 2

  源文第 2 页：https://docs.google.com/document/d/1SZAlExB1qAc9izHt54gwunNpjKE6wXb8Y7yA_e-baK8/edit?tab=t.0

- Source p. 35

  源文第 35 页：https://docs.pytorch.org/docs/stable/mps.html

- Source p. 35

  源文第 35 页：https://docs.pytorch.org/docs/stable/notes/mps.html

- Source p. 47

  源文第 47 页：https://doi.org/10.18653/v1/D19-1443

- Source p. 15

  源文第 15 页：https://einops.rocks/1-einops-basics/

- Source p. 15

  源文第 15 页：https://einx.readthedocs.io/en/stable/gettingstarted/basics.html

- Source p. 11

  源文第 11 页：https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character

- Source p. 33

  源文第 33 页：https://en.wikipedia.org/wiki/TensorFloat-32

- Source p. 45

  源文第 45 页：https://github.com/KellerJordan/modded-nanogpt

- Source p. 6

  源文第 6 页：https://github.com/openai/tiktoken/pull/234/files

- Source p. 17

  源文第 17 页：https://github.com/patrick-kidger/jaxtyping

- Source p. 2

  源文第 2 页：https://github.com/stanford-cs336/assignment1-basics

- Source p. 2

  源文第 2 页：https://github.com/stanford-cs336/assignment1-basics-leaderboard

- Source p. 8

  源文第 8 页：https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

- Source p. 47

  源文第 47 页：https://openreview.net/forum?id=Drrl2gcjzl

- Source p. 1

  源文第 1 页：https://pytorch.org/docs/stable/nn.html#containers

- Source p. 37

  源文第 37 页：https://wandb.ai